DISEASE-AWARE PADDY HARVEST YIELD ESTIMATION AND TREATMENT DECISION SUPPORT SYSTEM FOR SRI LANKAN PADDY CULTIVATION

Harshika M A B

(IT22261632)

BSc (Hons) degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DISEASE-AWARE PADDY HARVEST YIELD ESTIMATION AND TREATMENT DECISION SUPPORT SYSTEM FOR SRI LANKAN PADDY CULTIVATION

Harshika M A B

(IT22261632)

Dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: …… Date: ……

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the supervisor: ……. Date: ……

(Mrs. Chathurangika Kahandawarachchi)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my research supervisor, Ms. Chathurangika Kahandawarachchi, for her guidance, feedback and continued support throughout the development of this research component. I am also grateful to the Mr. Koliya Pulasinghe lecturer of the Faculty of Computing at the Sri Lanka Institute of Information Technology for providing the knowledge and research environment required to undertake this work.

We would also like to extend our sincere appreciation to our co-supervisor for the valuable guidance, knowledge, and constructive feedback provided throughout the research. The insights shared on agricultural practices, paddy disease identification, and system integration have helped us maintain the practical relevance of our research within the Sri Lankan agricultural context.

Finally, I would also like to acknowledge my group members for their cooperation and continuous support throughout the planning, development, testing, and documentation of the project. The collaborative efforts and knowledge shared among the team have been essential in overcoming challenges and successfully completing the components of this research.

We sincerely hope that the findings and outcomes of this research will contribute to the advancement of Smart Agriculture Decision Support System for the improvement of paddy cultivation in Sri Lanka.

ABSTRACT

Paddy cultivation in Sri Lanka is mainly affected by several diseases, including Rice Blast, Bacterial Leaf Blight, Narrow Brown Spot, and Brown Spot, which can reduce crop health and productivity. Early identification of these diseases and assessment of their severity is important for supporting timely agricultural decision-making. However, disease identification alone does not provide sufficient information about the extent of infection. This research presents a paddy disease detection and severity estimation component developed under Project R26-IT-066 using the YOLOv8 deep learning model.

The implementation involves paddy leaf image processing, YOLOv8-based model development, disease detection, affected-area-based severity estimation, and integration with the wider smart agriculture system. By combining leaf validation, health assessment, disease identification, and severity estimation in a single workflow, the component aims to provide more informative results than conventional disease identification approaches. The system's performance will be evaluated using the available dataset and experimental testing, considering disease detection performance, classification results, severity estimation, and response time. Overall, this research contributes a deep learning-based paddy disease analysis component designed to support early disease identification and severity-aware agricultural decision-making under Sri Lankan cultivation conditions.

Keywords: *Paddy disease detection; Rice Blast; Brown Spot; disease severity; deep learning; computer vision; smart agriculture; Decision Support System; Sri Lankan agriculture.*

TABLE OF CONTENTS

Declaration.....3
Abstract.....4
Acknowledgement.....5
Table of Contents….....6
1. Introduction 10
1.1 Background 11
1.2 Literature Review…….....12
1.3Research Problem….....14
1.4 Research Gap …….....15
1.5 Research Aim …….....16
1.6 Research Objectives
1.6.1 Main Objective 16
1.6.2 Specific Objectives 16
2. Methodology
2.1 Overview.....19
2.1.1System Architecture……21
2.1.2 Data Collection and Dataset Preperation…….....21
2.1.3 Image Processing……...27
2.1.4 YOLOv8-Based Disease Detection……...30
2.1.6 Stage-Based Assessment.……..32
2.1.7 Severity Decision-Making Framework…….34
2.1.8. App-Based System Implementation and Application Integration……37
2.2 Testing and validation
2.2.1 YOLOv8s Model Validation…….39
2.2.2 Hybrid Framework Validation……...40
2.3 Project Requirements

2.3.1 Functional Requirements……..41
2.3.2 Non-Functional Requirements……..43
2.3.3 System Requirements……43
2.4. Ethical and Security Considerations
2.4.1 Data Privacy and Protection……..45
2.4.1 Data Privacy and Protection……..45
2.5. Commercialization Aspects of the Product
2.5.1 Target Audience……46
2.5.2 Market Space……47
2.5.3 Potential Revenue Models……48
2.5.4 Commercialization Challenges……..49
3. RESULTS AND DISCUSSION
3.1 Results
3.1.1 Dataset Results…….…….49
3.1.2 YOLOv8s Detection Results…….…....51
3.1.3 Integrated Detection–Classification Pipeline Results…….……..56
3.1.4 Integration Results…….……59
3.3 Research Findings……60
3.4 Summary of Each Student's Contribution……60
3.5 Discussion……62
4. CONCLUSION AND FUTURE WORK
4.1 Conclusion……63
4.2 Future Work……..64
5. References…….…… 65

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DSS	Decision Support System
DL	Deep Learning
DoA	Department of Agriculture Sri Lanka
IRRI	International Rice Research Institute
IoT	Internet of Things
ML	Machine Learning
NPK	Nitrogen, Phosphorus and Potassium
REST	Representational State Transfer
API	Application Programming Interface
SLIIT	Sri Lanka Institute of Information Technology
SES	Standard Evaluation System
SST	Software Systems and Technologies

LIST OF FIGURES

Figure 2.1. Overall workflow of the paddy leaf disease-detection component……22

Figure 2.2. System architecture of the disease-detection component……....24

Figure 2.3. Rice Blast……....26

Figure 2.4 Bacterial Blast……..26

Figure 2.5 Brown Spot……..26

Figure 2.6 Narrow Brown…….26

Figure 2.7 Diseased leaf detection annotation with bounding boxes in YOLOv8 model ……....27

Figure 2.8 Healthy leaf detection annotation with bounding boxes in YOLOv8 model……....27

Figure 2.9. Image-input processing and inference preparation…….29

Figure 2.10 YOLOv8 inference and disease-result generation workflow…….32.

Figure 2.11 Disease assessment flow…….34

Figure 2.12 Spread-area-based severity decision-making framework……...37

Figure 2.13 Sequence diagram of the image-prediction request and response……..39

Figure 3.1: Class distribution and bounding-box characteristics of the paddy leaf dataset……...53

Figure 3.2: YOLOv8s training and validation metrics across training epochs……..53

Figure 3.3: Precision–recall curves for the displayed paddy leaf classes……..53

Figure 3.4: F1-score against confidence threshold……55

Figure 3.5: Precision against confidence……...55

Figure 3.6: Recall against confidence threshold……55

Figure 3.7: Confusion matrix of YOLOv8s class predictions……56

Figure 3.8: Normalized confusion matrix of YOLOv8s class predictions…….57

Figure 3.9: Examples of YOLOv8s predictions on paddy leaf images…….58

Figure 3.10: Example disease detections with predicted class labels and bounding boxes……...58

Figure 3.11. Detection page with image upload, disease results with severity grades……..60

LIST OF TABLES

Table 1.1: . Comparison of Existing Systems and the Proposed Component. 13

Table 1.2: Main components of the paddy leaf disease-detection architecture. 15

Table 2.1. Main components of the paddy leaf disease-detection architecture……..21

Table 2.2. Dataset categories relevant to the component...…….…… 24

Table 2.3 Summary of the image-processing operations…….. 29

Table 2.4 Outputs of the disease-assessment component……...……34

Table 2.5 Severity thresholds……..…….…… 35

Table 2.6. Test Scenario Validation ……..40

Table 2.7. Functional Requirements……...42

Table 2.8. Hardware Components …….44

Table 2.9. Software Components ……...44

Table 2.10. Revenue Models …….48

Table 3.1. Data set Results…….49

Table 3.2: Per-class AP values ……..52

Table 3.3. Severity Grades …….58

1. INTRODUCTION

1.1 Background and Context

Rice is a staple food and a major agricultural crop in Sri Lanka. Paddy farmers face production risks from pests, weather variability, soil conditions, and diseases. Leaf symptoms are often among the first visible indicators that a crop may be affected. The ability to screen leaf images quickly can help farmers and agricultural extension officers decide whether closer inspection or specialist advice is needed. The proposal for project R26-IT-066 positions this component within a wider decision-support system that combines field data, crop recommendations, disease detection, and harvest estimation.

This research component focuses on the development of a YOLOv8-based paddy leaf disease detection system using OpenCV for image processing and handling. The YOLOv8 object-detection approach can identify disease-related regions and associate them with predicted classes and confidence scores. This capability provides a basis for presenting both the predicted disease category and the approximate locations of visible symptoms in an image.

The implemented component targets four paddy leaf disease categories: Rice Blast, Bacterial Leaf Disease, Brown Spot, and Narrow Brown Spot. The system processes an uploaded leaf image, applies the trained YOLOv8 model to detect the supported disease categories, and uses the resulting predictions to generate an output for the application. OpenCV supports image handling and processing within the computer-vision workflow, while the trained detection model performs disease-related inference.

In addition to disease detection, Severity estimation introduces an additional challenge. The presence of a disease does not necessarily indicate how extensively the visible leaf surface is affected. A severity-related output can provide additional information, but its accuracy depends on how the affected area is measured and how severity categories are defined. In the current implementation, severity is derived from the area of detected bounding boxes relative to the complete image area. This is a practical approximation, but it is not equivalent to pixel-level lesion segmentation.

The proposed system is part of the broader project R26-IT-066, which aims to develop a smart agriculture decision-support system for Sri Lankan crops. By combining image-based disease detection with severity estimation, the paddy leaf component is intended to provide accessible information that may support crop monitoring and decision-making. The reliability of the system must be assessed through appropriate testing on representative paddy leaf images, including healthy leaves, diseased leaves, and images that do not contain paddy leaves.

1.2 Literature Review

Plant disease detection using computer vision has attracted considerable research attention because image-based techniques can support the identification of visible symptoms. Traditional image-processing approaches generally rely on manually selected features, such as colour distributions, texture characteristics, and lesion shapes. Although these methods can be useful in controlled conditions, their performance may be affected by changes in lighting, background, camera quality, and symptom appearance.

Deep learning reduces the need to define all visual features manually by learning feature representations from labelled training images. Mohanty et al. demonstrated the potential of deep-learning models for image-based plant disease identification, while Ferentinos investigated deep-learning approaches for plant disease detection and diagnosis [6], [7]. These studies provide a foundation for applying neural networks to agricultural image analysis. Nevertheless, performance obtained from one dataset does not automatically demonstrate equivalent performance in another crop, geographic region, or field environment.

Object detection is a related computer-vision task that identifies objects or regions within an image and predicts their locations using bounding boxes. The YOLO family of models is designed to perform object detection efficiently, making it suitable for applications where predictions must be generated within a practical timeframe. The YOLOv8 implementation provided by Ultralytics supports object-detection workflows and can be used with trained model weights to produce class predictions, confidence scores, and bounding-box coordinates [8].

In this component, YOLOv8 is used to detect four selected paddy leaf disease categories rather than relying only on whole image classification. This allows the output to include spatial information about the detected regions. OpenCV complements the detection process by providing image processing and image-handling functionality. Together, these technologies form the computer-vision foundation of the implemented disease-detection component.

Severity estimation introduces an additional challenge. The presence of a disease does not necessarily indicate how extensively the visible leaf surface is affected. A severity-related output can provide additional information, but its accuracy depends on how the affected area is measured and how severity categories are defined. In the current implementation, severity is derived from the area of detected bounding boxes relative to the complete image area. This is a practical approximation, but it is not equivalent to pixel-level lesion segmentation. Consequently, the severity categories require further validation against appropriately labelled images or expert assessments before they can be considered reliable measures of actual disease severity.

Overall, the literature demonstrates that deep learning is a promising approach to plant disease recognition from images. CNN-based studies have established the feasibility of learning disease-related visual features, while object-detection models provide additional information about the approximate locations of detected regions [6][7]. OpenCV can support image handling and processing, while YOLO provides the learned object-detection capability. Integrating these technologies with a Flutter client and Flask backend makes it possible to deliver image-based predictions through an application interface. Nevertheless, several challenges remain, including the representativeness of the training dataset, the ability to distinguish visually similar disease categories, the reliability of predictions under local field conditions, and the accuracy of severity estimates derived from bounding boxes. The present component addresses a practical part of this problem by providing a YOLO-based detection workflow for four paddy leaf disease categories and an approximate affected-area-based severity output. Its contribution must be established through verified experimental results, class-specific evaluation metrics, and testing of the complete application workflow. These evaluations are necessary to determine the reliability and practical suitability of the component for the intended agricultural decision-support context.

Table 1. 1 . Comparison of Existing Systems and the Proposed Component.

Study / system	**Method used**	**Main contribution**	**Limitation or research gap**
Mohanty et al. (2016) [1]	CNN architectures, including AlexNet and GoogLeNet	Demonstrated deep learning for image-based plant disease identification, with 99.35% accuracy on the controlled test set.	Performance was much lower on images from different conditions, of real-world images.

Ferentinos (2018) [2]	Deep learning models based on CNNs	Investigated deep learning for automated plant disease detection and diagnosis; the best reported model achieved 99.53% accuracy on the study's test images.	Broad crop coverage does not by itself demonstrate validated performance for Sri Lankan paddy fields or affected-area severity estimation.
Traditional image-processing approaches	Image segmentation, colour features, texture features and rule-based classification	Can identify visual differences associated with disease symptoms and may be computationally straightforward.	Performance can depend on background, image quality and manually selected features, disease classification and severity estimation.
CNN-based disease classification systems	CNN feature extraction and classification	Learn visual patterns associated with disease symptoms without requiring all features to be manually designed.	A disease label alone does not necessarily quantify affected area or provide a severity estimate.
Proposed IT22261632 component	**YOLO-based detection using a trained model, with OpenCV/image processing in the application pipeline**	**Designed to return disease-related information, prediction confidence and an affected-area-based severity estimate, as part of the wider R26-IT-066 system.**	**Performance on Sri Lankan field images, paddy-leaf validation, severity-estimation accuracy and end-to-end reliability must be demonstrated through testing.**

1.3 Research Problem

Existing disease detection approaches may identify a crop disease without providing sufficient information about its severity or the urgency of intervention. A farmer who receives only a label such as 'Rice Blast detected' still needs to determine whether the infection is mild or severe, whether immediate action is required, and how the result should influence subsequent cultivation and harvest decisions.

The individual research problem needs for a localized disease analysis component that can identify important paddy diseases from leaf images, estimate disease severity, and provide outputs that are useful for decision support in Sri Lankan paddy cultivation.

1.4 Research Gap

Existing research has demonstrated the potential of deep learning and computer vision for plant disease classification and detection. However, several aspects remain important when developing a practical paddy disease detection system. Image classification methods generally provide an image-level disease label but do not directly indicate the location of the affected region. Object detection methods, such as YOLO-based approaches, can localize detected regions using bounding boxes, but these boxes may not precisely represent the actual boundaries of diseased tissue.

Another important consideration is the estimation of disease severity. Detecting a disease category does not necessarily indicate how much of a leaf is affected. Estimating the proportion of the image covered by detected bounding boxes can provide an approximate measure, but its accuracy depends on the detection results and the relationship between the bounding boxes and the actual lesions. Consequently, the reliability of severity estimation requires separate consideration from disease classification performance.

The proposed contribution addresses these gaps through a disease detection and severity analysis pipeline localized to Sri Lankan paddy cultivation. The proposal identifies the disease classes and defines a four-level severity representation.

Table 1.2: Main components of the paddy leaf disease-detection architecture.

Area	**Existing/Reviewed Approaches**	**IT22261632 Contribution**
Disease identification	Disease classification is commonly provided.	Localized paddy disease analysis for the selected classes.
Severity	Often absent or limited.	Four-level severity representation proposed.
Local context	Many datasets are not Sri Lankan field datasets.	Sri Lankan field-oriented dataset and localization are proposed.

Actionable output	Disease label may be the final output.	Severity-aware management support is proposed.
System integration	Many systems operate independently.	Designed as a component of R26-IT-066.

1.5 Research Aim

The aim of this research is to design and implement an AI-based paddy disease detection and severity analysis component that supports timely, severity-aware decision making for Sri Lankan paddy cultivation.

1.6 Research Objectives

1.6.1 Main Objective

The main objective is to design, develop, and evaluate a YOLOv8-based paddy leaf disease detection system that analyses uploaded leaf images to identify whether the input represents a paddy leaf, determine whether the leaf is healthy or unhealthy, and classify unhealthy leaves into four target disease categories: rice blast, bacterial leaf disease, brown spot, and narrow brown spot. In addition to disease identification, the system aims to estimate the extent of visible disease spread based on the detected regions, determine a corresponding disease severity level, and present the predicted disease name, model confidence score, estimated spread-area percentage, and severity information through an integrated application interface. The overall purpose is to provide users with an accessible image-based screening tool that supports the early identification and preliminary assessment of paddy leaf diseases.

1.6.2 Specific Objectives

- **Identify and validate paddy leaf images**

To develop an image-validation process that determines whether an uploaded image can be reliably recognized as a paddy leaf image before presenting a disease prediction. This objective focuses on reducing incorrect predictions caused by unrelated images, unclear photographs, poor lighting, or unsuitable image quality. The system should use model prediction and confidence information to identify uncertain inputs and communicate when an image should be uploaded again. A dedicated paddy-leaf versus non-paddy-leaf classification mechanism should be evaluated if explicit input validation is required.

- **Distinguish healthy and unhealthy paddy leaves**

To determine whether an identified paddy leaf shows signs of a healthy condition or a potential disease. This objective aims to use the visual features detected by the YOLOv8 model to distinguish disease-related symptoms from images classified as healthy. The system should present the predicted condition clearly and avoid treating uncertain or failed detection as definitive evidence that the leaf is healthy.

- **Detect and classify four paddy leaf diseases using YOLOv8**

To implement a YOLOv8-based image-analysis mechanism for identifying the four target diseases.

- Rice blast: Identification of visual symptoms associated with rice blast disease.
- Bacterial leaf disease: Identification of the bacterial leaf disease category represented in the labelled dataset and trained model.
- Brown spot: Identification of visual symptoms associated with brown spot disease.
- Narrow brown spot: Identification of visual symptoms associated with narrow brown spot disease.

The objective is to return the predicted disease category and its associated model confidence value for an unhealthy paddy leaf. The model's class names and class-index mapping should be verified to ensure that the deployed weights correctly represent all four target categories.

- **Estimate the extent of visible disease spread**

To estimate the proportion of the image area occupied by the regions detected by the YOLOv8 model. This objective aims to provide additional information about the visible distribution of disease symptoms rather than reporting only a disease name. The estimated spread-area percentage is calculated using the dimensions of the detected regions relative to the image dimensions. The calculation method and its limitations should be documented because bounding-box areas provide an approximation of affected areas and may not precisely represent the actual diseased leaf tissue.

- **Determine the severity level of the detected disease**

To categorize the estimated disease spread into a severity level using the thresholds defined in the implemented system. This objective aims to translate the estimated spread-area percentage into an understandable severity category, allowing users to distinguish between different extents of visible symptoms. The system currently maps estimated spread to the categories Mild, Moderate, High, and Severe, while healthy predictions receive no disease-severity label. The reliability of these

categories should be assessed against appropriately labelled reference images before they are interpreted as validated agricultural severity ratings.

- **Provide prediction, confidence and interpretable results**

To display the predicted disease name, confidence percentage, estimated spread-area percentage, and severity level in a clear and understandable format. This objective aims to help users interpret the model output and recognize when a prediction may be uncertain. The confidence percentage should be presented as the model's confidence for the relevant detection, rather than as the overall accuracy of the system. The result should also be distinguished between a valid prediction, an uncertain input, and a healthy-leaf prediction.

- **Integrate the disease detection model with the application interface**

To integrate the YOLOv8 inference mechanism with the backend API and the Flutter-based user interface so that users can submit leaf images and receive prediction results. The backend should process uploaded images, perform model inference, calculate the estimated spread area, determine the severity category, and return the results in a structured response. The frontend should display the returned information consistently and communicate errors or uncertain predictions appropriately.

- **Evaluate the functionality and predictive performance of the system**

To evaluate the developed component through functional testing and quantitative model evaluation. Functional testing should examine image uploads, prediction responses, confidence handling, severity calculation, invalid inputs, and the display of results. Model performance should be measured using an independent labelled test dataset and appropriate metrics, including accuracy, precision, recall, F1-score, and a confusion matrix. Where suitable ground-truth bounding-box annotations are available, object-detection metrics such as mean Average Precision (mAP) should also be considered. The evaluation should identify classification errors, limitations in disease-spread estimation, and areas requiring further improvement.

METHODOLOGY

2.1 Overview

The methodology adopted for the IT22261632 component focuses on developing an image-based paddy leaf disease detection system to support the identification of diseases affecting rice cultivation in Sri Lanka. This component forms part of the broader project, Smart Agriculture Decision Support System for Sri Lankan Crops Using IoT, Machine Learning, and Image Processing, under project identifier R26-IT-066.

The primary purpose of the component is to analyze an uploaded leaf image and provide information about the apparent health condition of the leaf. The component focuses on four target diseases: Rice Blast, Bacterial Leaf Disease, Brown Spot, and Narrow Brown Spot.

The proposed workflow consists of several sequential stages. First, an image is submitted to the system and assessed to determine whether it can be recognized as a relevant paddy leaf image. Next, the system evaluates the leaf's health condition. If the leaf is identified as unhealthy, the disease-detection model predicts the corresponding disease class. Finally, the system calculates an estimated disease-spread percentage, determines a severity category according to the configured thresholds, and presents prediction confidence and associated information to the user.

The component uses the YOLOv8 object-detection framework through the Ultralytics Python library. The backend is implemented using Flask, while the user interface is implemented using Flutter. The backend processes uploaded images, obtains model predictions, estimates spread area from predicted bounding boxes, and returns the result through a JSON-based API.

The methodology is organized into seven main areas: system architecture, data collection and dataset preparation, image processing, YOLOv8-based disease detection, disease-stage assessment, severity decision-making, and application implementation.

The workflow represents the intended functionality. The system implements confidence-based handling, disease prediction, spread estimation, and severity mapping. However, the current confidence-based input check is a heuristic and should not be described as a separately trained paddy-leaf classifier.

No table of figures entries found.

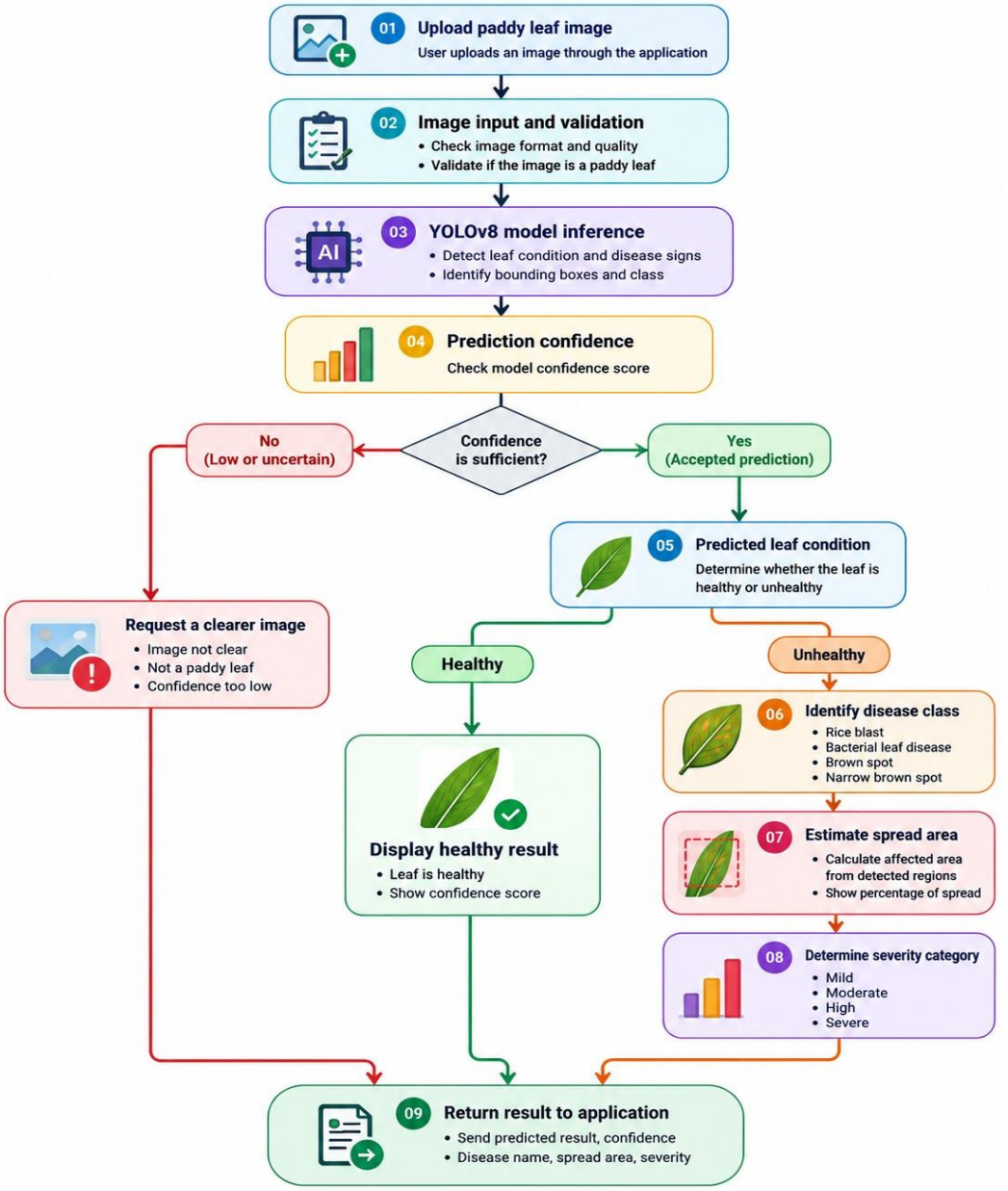


Figure 2.1. Overall workflow of the paddy leaf disease-detection component.

2.1.1 System Architecture

The system architecture defines the interaction between the user interface, backend API, YOLOv8 model, and result-presentation component. The architecture is designed to separate image submission, model inference, result processing, and user-interface responsibilities.

The system is implemented with a Flutter-based frontend and a Python Flask backend. The Flutter interface allows the user to select a leaf image and initiate the disease-detection process. The selected image is submitted to the Flask API through an HTTP request. The API passes the image to the YOLOv8 model, processes the prediction output, estimates the disease-spread percentage, and determines the severity category.

The backend returns a JSON response containing the prediction status, disease name, confidence percentage, spread-area percentage, and severity label. The Flutter interface then interprets the response and presents the information to the user.

The disease-detection API is implemented as a Flask blueprint registered under the /disease prefix. The prediction route is /predict, making the combined endpoint /disease/predict when the main Flask application is running.

Table 2.1. Main components of the paddy leaf disease-detection architecture.

Component	**Technology**	**Responsibility**
User interface	Flutter and Dart	Image selection, prediction initiation, and result presentation
API communication	HTTP and JSON	Transfers images and prediction results
Backend service	Python and Flask	Validates requests and coordinates inference
Image processing	OpenCV	Reads uploaded images and obtains image dimensions
Disease-detection model	YOLOv8 through Ultralytics	Predicts disease classes, confidence values, and bounding boxes
Severity assessment	Python logic	Estimates spread percentage and maps it to a severity label
Model weights	best.pt	Stores the trained model parameters used for inference

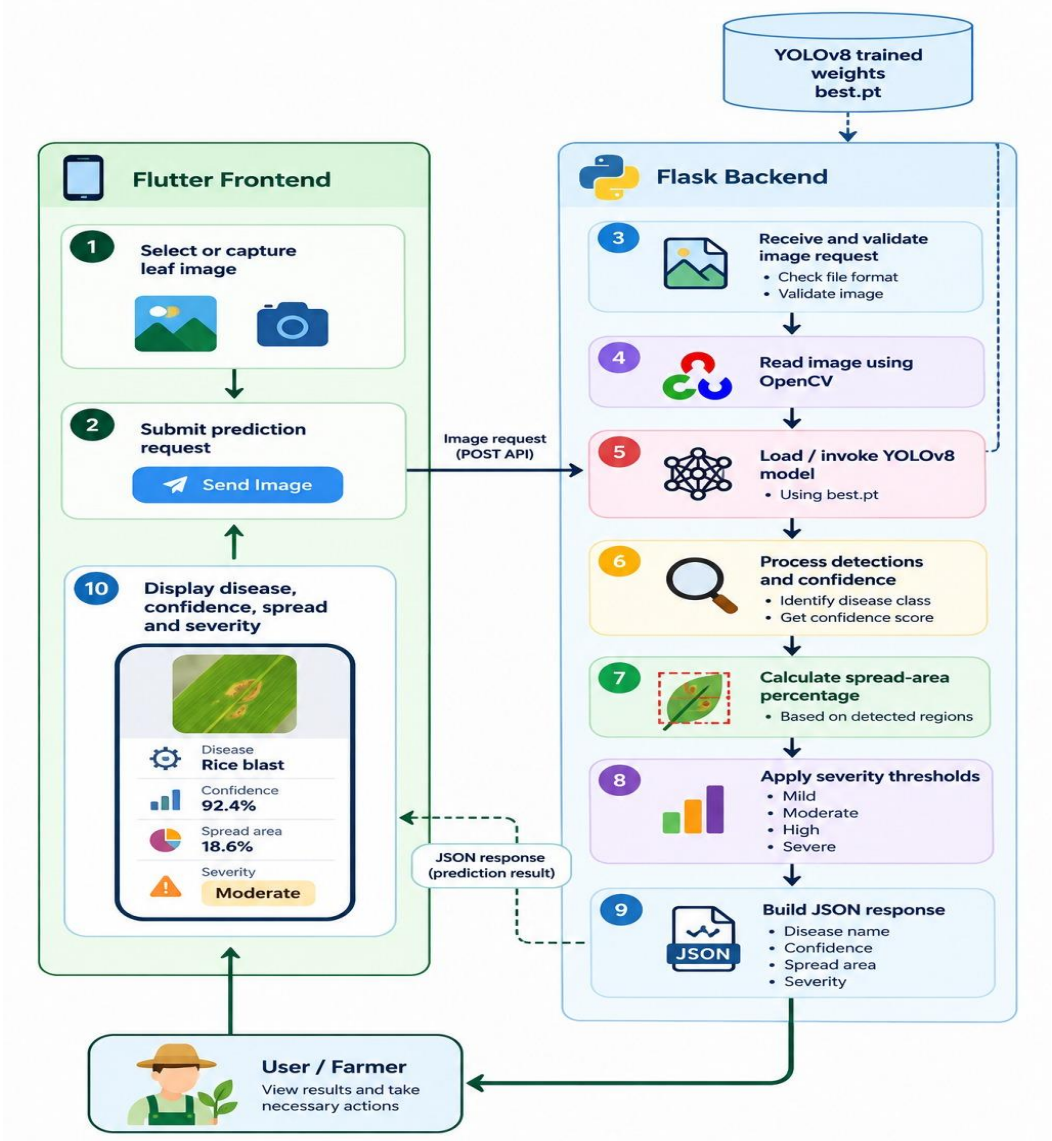


Figure 2.2. System architecture diagram.

The figure illustrates the principal request-and-response flow. The trained model weights are loaded by the backend, and the prediction process is performed when an image is submitted. The severity assessment operates on the detection results returned by the model.

The architecture also allows the disease-detection component to be integrated with the decision-support system. However, the present methodology evaluates only the individual disease-detection component and its application interface.

The system follows the steps below.

1. The user selects a paddy leaf image through the Flutter interface.

2. The frontend sends the image to the Flask prediction endpoint.
3. The backend reads the image and submits it to the YOLOv8 model.
4. The model returns the predicted class, confidence value, and detected bounding boxes.
5. The backend applies confidence handling and processes the detected regions.
6. The system estimates the spread-area percentage and assigns a severity label.
7. The API returns the result as JSON.
8. The Flutter interface displays the prediction and its associated information.

This workflow provides a modular structure in which image processing, inference, severity calculation, and result presentation can be tested separately.

2.1.2 Data Collection and Dataset Preparation

Data Collection

The performance of an image-based disease-detection system depends on the quality, diversity, and accuracy of the images used to train and evaluate the model. The dataset should represent the visual characteristics of healthy paddy leaves and leaves affected by the target diseases.

For this component, the required disease categories are Rice Blast, Bacterial Leaf Disease, Brown Spot, and Narrow Brown Spot. Healthy paddy leaf images and non-paddy images are also important for evaluating the healthy/unhealthy decision and the reliability of input validation.

Rice Blast commonly produces lesions with characteristic spindle-shaped or elliptical patterns. Brown Spot can produce brown lesions of varying sizes, while bacterial leaf diseases may produce elongated lesions or leaf discolorations. Narrow Brown Spot is associated with narrow, elongated brown lesions. Since different diseases can produce visually similar symptoms, the dataset should contain sufficient examples of each class to support reliable discrimination [1], [2].



Figure 2.3 Rice Blast



Figure 2.4. Bacterial Blast



Figure 2.5 Brown Spot



Figure 2.6 Narrow Brown

The implementation archive includes trained model weights and image files, but these alone do not establish the complete original training dataset, its size, the sources of all images, or the exact train-validation-test split. Consequently, the final dissertation should report those details only after they have been verified from the actual training records.

The table describes the intended evaluation categories. The presence of a separate healthy class or a dedicated non-paddy class in the trained model must be confirmed by inspecting the model's actual class mapping.

Table 2.2. Dataset categories relevant to the component.

Category	**Purpose**	**Expected system behavior**
Healthy paddy leaf	Establish normal leaf appearance	Return a healthy result if supported by the model

Rice Blast	Identify rice blast symptoms	Predict Rice Blast
Bacterial Leaf Disease	Identify bacterial leaf symptoms	Predict the appropriate bacterial disease class
Brown Spot	Identify brown spot symptoms	Predict Brown Spot
Narrow Brown Spot	Identify narrow brown spot symptoms	Predict Narrow Brown Spot
Non-paddy or unrelated images	Test input-validation reliability	Reject or flag uncertain inputs rather than confidently assigning a rice disease

Dataset Preparation

Dataset preparation involves organizing the available images, verifying their labels, and preparing them for model training and evaluation. Each image should be associated with a reliable class label. Where object detection is used, the diseased regions should also have appropriate bounding-box annotations.

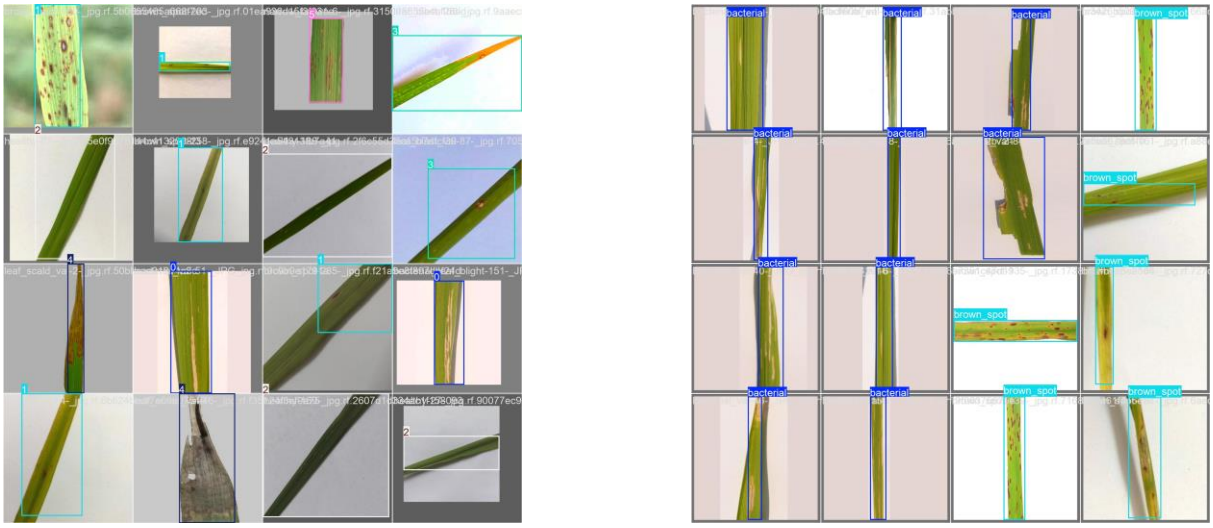


Figure 2.7 Diseased leaf detection annotation with bounding boxes in YOLOv8 model

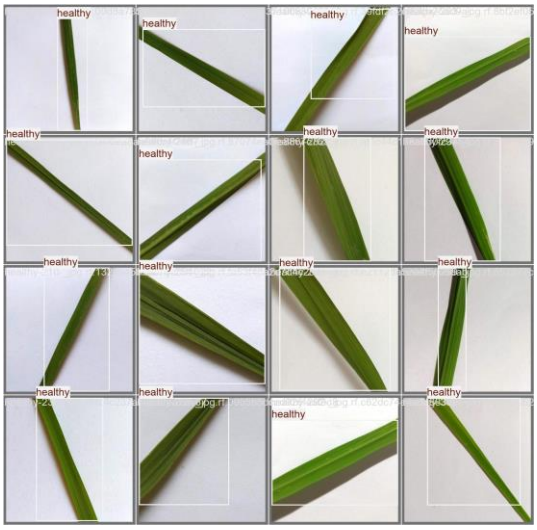


Figure 2.8 Healthy leaf detection annotation with bounding boxes in YOLOv8 model

The preparation process should include the following activities:

- Image verification: Remove unreadable files, duplicate images, and images that do not contain useful visual information.
- Label verification: Check that each image and its bounding-box annotations correspond to the correct disease category.
- Class organization: Organize the dataset according to the verified class names used by the trained YOLOv8 model.
- Annotation preparation: Ensure that object-detection annotations follow the format required by the selected YOLO training workflow.
- Dataset splitting: Separate the images into training, validation, and testing subsets.
- Class-distribution analysis: Record the number of images and labelled objects in each category to identify class imbalance.
- Data-leakage prevention: Keep duplicate or near-identical images from appearing in both training and testing subsets.

The training set is used to learn model parameters, the validation set supports model and configuration selection, and the independent test set is used to estimate generalization performance.

Dataset Quality Considerations

The images should represent variations in lighting, background, camera distance, disease severity, and leaf orientation. A dataset containing only clear images captured under controlled conditions may not represent the images submitted by farmers in real agricultural environments.

Where images are collected from multiple sources, the source and permitted usage of each dataset should be recorded. Expert review of a sample of the labels is recommended because incorrect annotations can affect both disease classification and severity estimation.

These preparation steps help establish a more reliable basis for evaluating the model's ability to distinguish the four target diseases.

2.1.3 Image Processing

Image processing prepares the uploaded image for inference and extracts the visual information needed by the disease detection model. In the implemented system, the backend receives the uploaded image and uses OpenCV to read it and obtain its dimensions. The image is then passed to the YOLOv8 model for inference.

Image Input and Validation

The process begins when the user selects an image through the application. The frontend submits the image using the multipart form field named image. The backend checks whether an image file is present and whether the uploaded filename is empty before proceeding.

After receiving the image, the backend saves it temporarily and reads it using OpenCV. The image dimensions are used in the later calculation of the estimated spread-area percentage.

The application also uses the model's confidence output to decide how the result should be presented. In the current implementation, the backend applies a 50% confidence threshold, whereas the Flutter interface uses a 55% threshold to identify low-confidence inputs. This difference should be corrected before the final deployment because the same prediction can be interpreted differently by the backend and frontend.

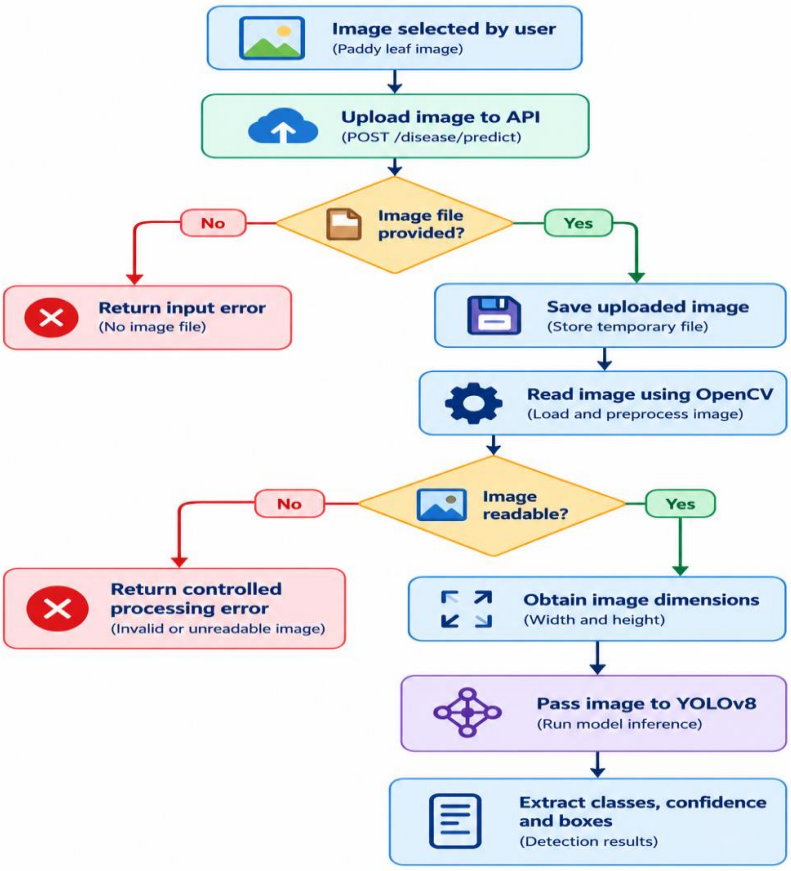


Figure 2.9. Image-input processing and inference preparation.

The diagram distinguishes the intended error-handling workflow from the image-processing operations verified in the source. Explicit validation of unreadable images should be tested to ensure that malformed files produce controlled errors rather than unhandled failures.

Bounding-Box Processing

YOLOv8 can return one or more bounding boxes for detected regions. Each bounding box is represented by its upper-left and lower-right coordinates:

$$B_i = (x_{1i}, y_{1i}, x_{2i}, y_{2i})$$

The width and height of a box are calculated as:

$$w_i = x_{2i} - x_{1i}$$

$$h_i = y_{2i} - y_{1i}$$

The area of each bounding box is:

$$A_i = w_i \times h_i$$

The image area is calculated using its width and height:

$$A_{\text{image}} = W \times H$$

In the backend, the areas of the detected boxes are summed and divided by the total image area to estimate the percentage of the image covered by the detected regions.

This is an approximate bounding-box coverage measure. It is not a direct measurement of the percentage of diseased leaf tissue because the boxes may include healthy leaf pixels or background, and overlapping boxes may be counted more than once.

Image Processing Considerations

Table 2.3 Summary of the image-processing operations.

Operation	**Purpose**
Image upload	Transfer the selected image to the backend
File-presence check	Detect missing or empty uploads
Image reading	Load the uploaded image using OpenCV
Image-dimension extraction	Obtain image width and height
Model inference	Generate detections from the image
Bounding-box extraction	Obtain predicted region coordinates
Image resizing and augmentation	Prepare images for training, if configured
Leaf-background separation	Restrict area calculations to the leaf
Diseased-pixel segmentation	Measure the actual affected tissue

The distinction between implemented operations and potential improvements is important because image resizing, augmentation, leaf segmentation, and lesion segmentation should not be described as completed activities unless supported by the corresponding code or training configuration.

2.1.4 YOLOv8-Based Disease Detection

(YOLO) is an object-detection approach designed to predict object classes and their locations within an image. The YOLOv8 implementation used in this component is accessed through the Ultralytics Python package [6].

Unlike a basic image-classification model that produces a class for an entire image, an object detector can return bounding boxes, class predictions, and confidence scores for detected regions. These outputs allow the component to associate the disease prediction with a detected region and use the region coordinates in the spread-area estimation process.

The system implementation loads the trained model weights from backend-python/disease/best.pt. During inference, the backend submits the uploaded image to the model and retrieves the prediction results.

The disease detection process consists of the following operations.

- Model initialization

The YOLO model is loaded through the Ultralytics library using the trained weights file best.pt. The model provides the class-name mapping and prediction functionality required by the API.

- Image inference

After the backend reads the uploaded image, the image is submitted to the model. The model generates detection results containing bounding-box coordinates, predicted class indices, and confidence scores.

- Class identification

The backend obtains the class name associated with the first detected bounding box. The class-name mapping is provided by the loaded model.

The deployed class mapping must be checked to confirm that it corresponds to the required disease categories: Rice Blast, Bacterial Leaf Disease, Brown Spot, and Narrow Brown Spot. The source code alone does not establish that the trained weights contain all four classes under those exact names.

- Confidence extraction

The confidence score associated with the first detected bounding box is converted into a percentage and rounded to two decimal places for the API response.

The confidence score represents the model's confidence in that particular prediction. It must not be interpreted as the overall accuracy of the model.

- Detection-region processing

The bounding boxes returned by the model are processed to estimate the image area covered by the detections. This value is subsequently used by the severity decision-making logic.

- Result generation

The backend prepares a JSON response containing the prediction status, disease name, confidence percentage, estimated spread-area percentage, and severity label.

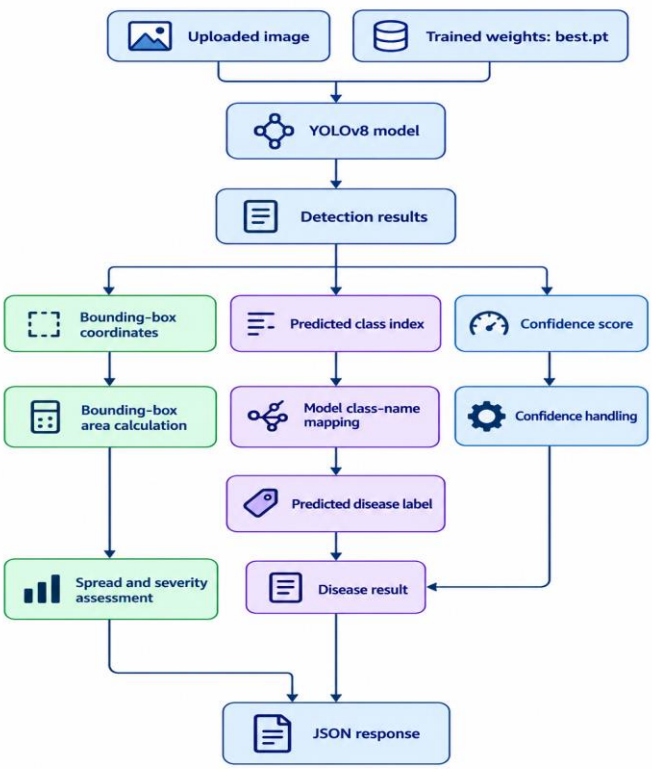


Figure 2.10 YOLOv8 inference and disease-result generation workflow.

The confidence score is used to determine whether a prediction should be accepted or flagged as uncertain. A threshold-based mechanism can reduce the number of low-confidence results presented as reliable predictions. However, the threshold alone cannot establish whether an image contains a paddy leaf.

The backend uses a confidence threshold of 0.50. When the first detection's confidence falls below this threshold, the backend changes the disease label to healthy and resets the estimated spread to zero. The Flutter interface separately uses a threshold of 55% to decide whether the input should be treated as an unrecognized rice leaf.

This behaviour creates an inconsistency between the backend and the interface. It can also result in a low-confidence image being treated as healthy by the backend even when the image is diseased.

A more reliable implementation should distinguish between the following states:

- **Input validity:** Whether the image is recognized as a paddy leaf.
- **Health condition**: Whether the leaf is healthy or unhealthy.
- **Disease class:** The predicted disease for an unhealthy leaf.
- **Prediction uncertainty:** Whether the model's confidence is sufficient to present a class prediction.

2.1.5 Stage-Based Assessment

Disease-stage assessment aims to interpret the model's output in terms of the apparent extent of disease symptoms. Identifying the likely disease is useful, but it does not independently communicate how widely visible symptoms are distributed across the image.

In the component, disease assessment combines the predicted disease class with the estimated spread-area percentage and a corresponding severity category. The approach uses the bounding-box regions returned by YOLOv8 as an approximation of the visible area associated with the detections.

It is important to distinguish disease-stage assessment from disease severity assessment. Disease stage may refer to progression over time or the biological development of a disease, whereas severity describes the extent of damage or symptoms observed at a particular time. The supplied code calculates a spread-based severity category; it does not establish a temporal disease-progression model.

The assessment workflow can be described in three main stages.

- Leaf-condition assessment

The image is processed by the model and the prediction confidence is examined. The application uses confidence-based handling to decide whether to present the prediction or flag the image as unrecognized.

- Disease identification

If the image produces an accepted unhealthy disease prediction, the system identifies the corresponding disease class. The four target classes are Rice Blast, Bacterial Leaf Disease, Brown Spot, and Narrow Brown Spot.

- Spread-based severity assessment

The system estimates the area covered by predicted bounding boxes and maps the resulting percentage to one of the configured severity categories. The disease label, confidence, spread estimate, and severity are then returned to the interface.

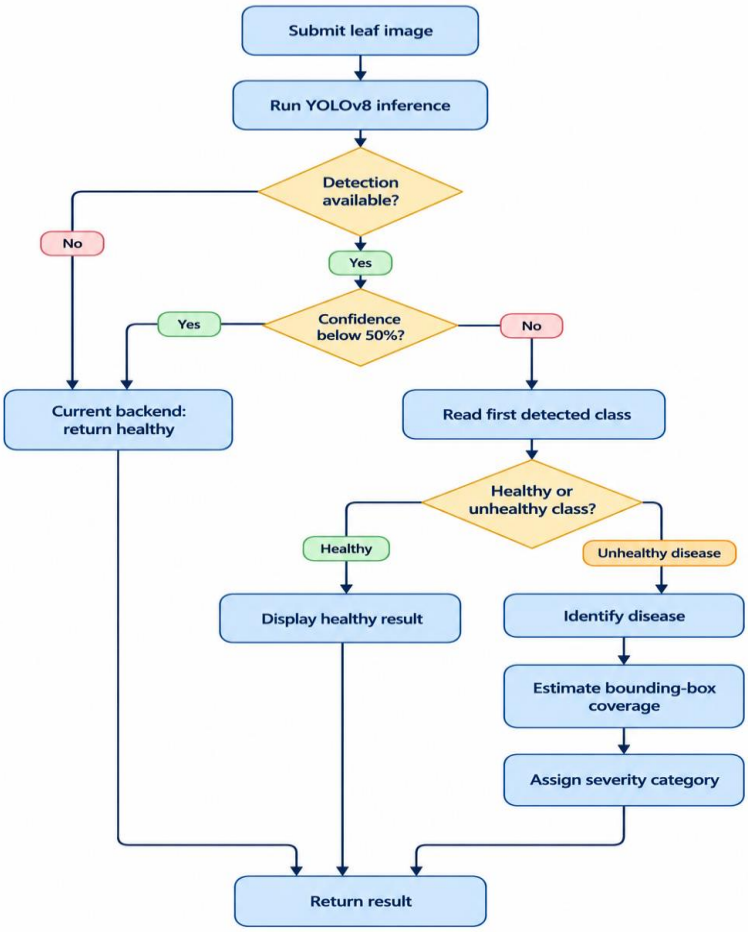


Figure 2.11 Disease assessment flow

The decision flow above deliberately documents the current backend behaviour. A final, improved workflow should not equate a missing detection with a confirmed healthy leaf. Instead, it should return an uncertain or unrecognized result unless healthy status has been positively established.

The spread-area estimate and severity label should be interpreted as model-derived indicators. They should not be treated as validated measurements of actual crop loss or as definitive agricultural advice.

Table 2.4 Outputs of the disease-assessment component.

Output	**Description**	**Purpose**
Input status	Indicates whether the prediction is accepted or requires a clearer image	Supports appropriate interpretation of the result
Disease label	Predicted disease class	Communicates the likely disease
Confidence percentage	Confidence associated with the selected detection	Indicates the model's confidence in the prediction
Spread-area percentage	Estimated coverage of detected bounding boxes relative to the image	Represents approximate spatial extent
Severity label	Mild, Moderate, High, Severe, or None, according to the implementation	Communicates the estimated severity category

2.1.6 Severity Decision-Making Framework

The severity decision-making framework converts the detected-region information into an estimated spread percentage. This calculation is implemented in the Flask backend.

Let A_i represent the area of the i-th predicted bounding box, N represents the number of detected boxes, and A_{image} represents the total image area.

The current implementation estimates the spread percentage as:

$$S_{\text{box}} = \frac{\sum_{i=1}^{N} A_i}{A_{\text{image}}} \times 100$$

where:

$$A_{\text{image}} = W \times H$$

Here, (W) and (H) are the width and height of the uploaded image.

This formula estimates the percentage of the full image covered by the predicted bounding boxes. Since it does not isolate the leaf from the background or segment individual diseased pixels, the result should be referred to as estimated bounding-box coverage or estimated spread-area percentage, rather than an exact measurement of diseased tissue.

The backend maps the calculated spread percentage to four severity labels. It also returns None for the severity when the final disease label is healthy.

Table 2.5 Severity thresholds.

Estimated Level	**Severity label**	**Interpretation**
0	Healthy	The backend returns a healthy result and zero spread
1	Mild	Low estimated detection coverage
2	Moderate	Intermediate estimated detection coverage
3	High	Relatively large, estimated detection coverage
4	Severe	Very large, estimated detection coverage

The severity-assessment procedure can be expressed as follows:

1. Obtain the uploaded image's width and height.
2. Calculate the total image area.
3. Extract the bounding-box coordinates returned by YOLOv8.
4. Calculate the area of each bounding box.
5. Sum the detected bounding-box areas.
6. Divide the summed area by the total image area and multiply by 100.
7. Apply the severity thresholds to the resulting percentage.
8. Return the estimated spread and severity alongside the disease prediction.

A simplified mathematical representation of the decision function is:

$$V(S) = \begin{cases} \text{Mild}, & 0 \leq S \leq 5 \\ \text{Moderate}, & 5 < S \leq 20 \\ \text{High}, & 20 < S \leq 40 \\ \text{Severe}, & S > 40 \end{cases}$$

where *(S)* is the estimated spread-area percentage. The healthy override takes precedence over this mapping and returns None for severity.

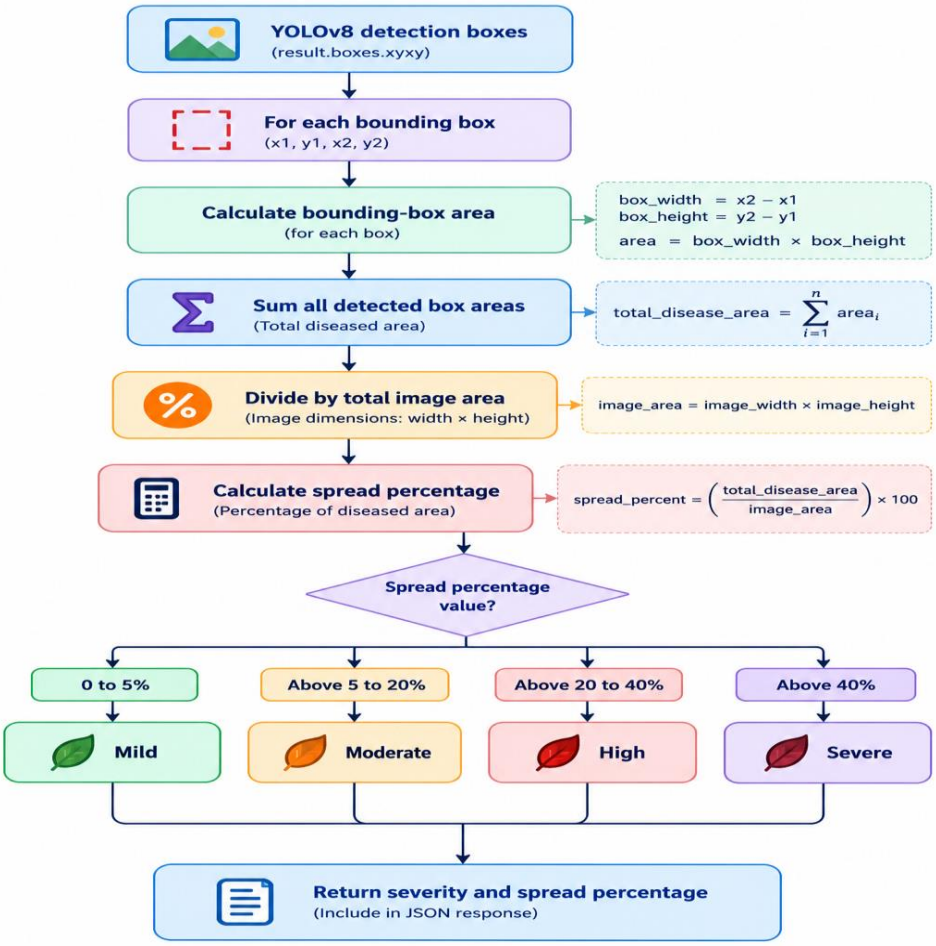


Figure 2.12 Spread-area-based severity decision-making framework.

The severity method has several limitations. First, a bounding box includes the entire rectangular region surrounding a detected object and may therefore include unaffected pixels. Second, overlapping boxes may cause parts of the image to be counted more than once. Third, the denominator is the total photograph area rather than the actual paddy-leaf area.

For these reasons, the severity label represents a preliminary estimate based on the detected image regions. A more precise future approach would first identify the leaf region and then use lesion segmentation to estimate the proportion of leaf pixels affected by disease. Expert-labelled reference

images could subsequently be used to evaluate the agreement between the calculated severity and an agricultural specialist's assessment.

2.1.7 App-Based System Implementation and Application Integration

The application implementation connects the disease-detection model with a user-facing interface through a backend API. The objective is to allow a user to submit a paddy leaf image and receive an understandable prediction without needing to interact directly with the machine-learning model.

Although this section is titled Web-Based System Implementation to match the requested methodology structure, the supplied project implementation uses a Flutter application and a Flask backend API. It should therefore be described as a client-server application unless a separate web frontend is also implemented. The implementation consists of three principal layers: the presentation layer, the backend processing layer, and the machine-learning inference layer.

The presentation layer is implemented using Flutter and Dart. The disease detection screen allows the user to select an image and initiate the scanning process. After receiving the response from the backend, the interface displays the prediction information. Depending on the response and confidence-handling logic, the screen presents the detected condition, disease severity, confidence percentage, and estimated infection spread. The interface is intended to make the output understandable to users who may not have technical knowledge of machine learning. Clear result labels and meaningful error messages are important because a low-confidence prediction should not be presented as a certain diagnosis.

The backend is implemented using Python and Flask. The disease functionality is organized as a Flask blueprint and registered under the /disease route prefix. The prediction endpoint accepts an image through an HTTP POST request. The uploaded image is read using OpenCV and passed to the YOLOv8 model. The backend then processes the detection results and generates the JSON response. The exact values returned depend on the model predictions and backend decision logic. The final dissertation should include an actual response captured during testing rather than an invented example.

The application follows a request-response pattern. When the user submits an image, the frontend sends the image to the prediction API. The backend executes the inference workflow and returns the result. The frontend then updates the screen using the response.

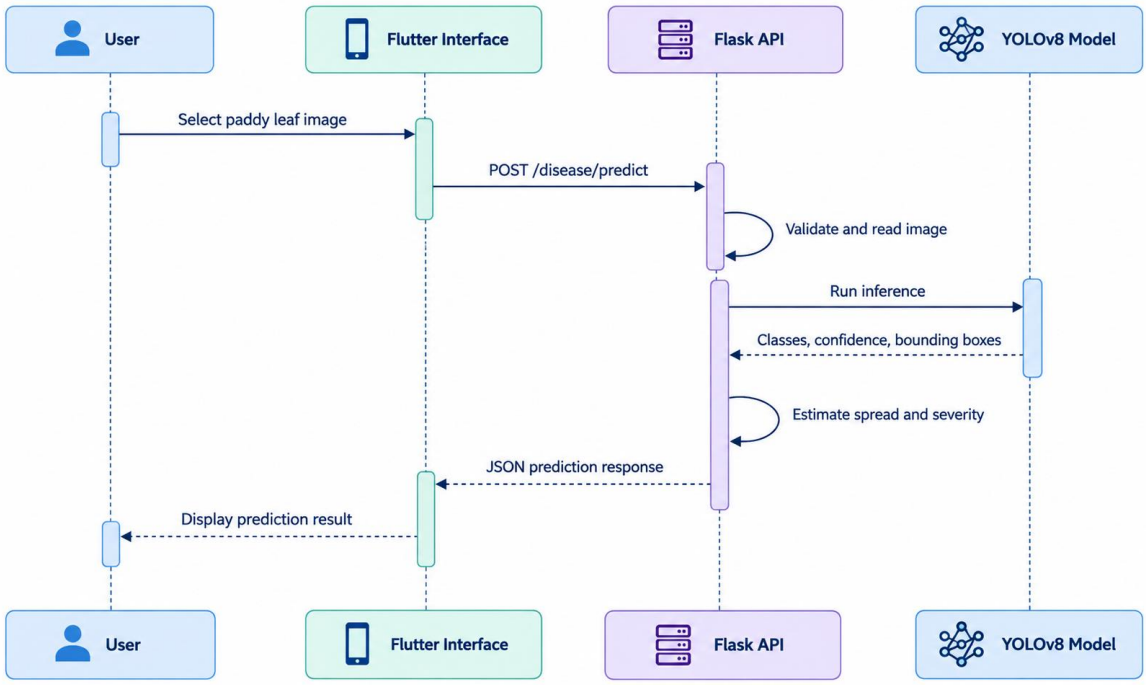


Figure 2.13 Sequence diagram of the image-prediction request and response

2.2 Testing and validation

Testing and validation are essential for ensuring that the proposed paddy disease detection component operates correctly, produces reliable predictions, and meets its functional requirements. The testing process focuses on paddy leaf image validation, healthy and unhealthy leaf classification, identification of four paddy diseases, disease severity estimation, and the integration of the component with the wider smart agriculture system.

The system should be evaluated using representative paddy leaf images containing healthy leaves, diseased leaves, non-paddy images, and images with varying degrees of disease spread. Testing should also consider variations in image quality, lighting conditions, leaf orientation, and background complexity. These tests help identify potential weaknesses in the system and assess its suitability for practical use under Sri Lankan paddy cultivation conditions.

The main testing activities include:

- Functional testing: Verifying that each system feature operates according to its intended requirements.
- Model testing: Evaluating the model's ability to identify paddy leaves, distinguish healthy and unhealthy leaves, and detect the relevant diseases.

- Severity estimation testing: Checking whether estimated severity corresponds to the proportion of the leaf area affected by disease.
- Integration testing: Verifying communication between the detection component, backend services, and user interface.
- Performance testing: Measuring prediction response time and resource consumption under expected operating conditions.
- Error-handling testing: Checking how the system responds to invalid images, unsupported file formats, missing inputs, and processing failures.

Validation determines whether the developed component meets its intended objectives and produces meaningful outputs for users. The validation process focuses on the YOLOv8 model and the complete disease analysis workflow.

2.2.1 YOLOv8s Model Validation

The YOLOv8s model is used as the deep learning approach for the paddy leaf image analysis component. The model validation process evaluates its ability to identify relevant paddy leaf content and support the subsequent disease analysis workflow.

Validation should use a separate test dataset that was not used to train or tune the model. The test dataset should contain representative examples of paddy leaves, healthy leaves, leaves affected by the four target diseases, and non-paddy images where applicable.

The validation process should cover the following areas:

- **Paddy leaf validation:** Assessing whether the system correctly distinguishes paddy leaf images from images that do not contain paddy leaves.
- **Healthy and unhealthy identification:** Evaluating whether the system correctly identifies healthy paddy leaves and recognizes leaves showing disease symptoms.
- **Disease identification:** Measuring the model's ability to identify Rice Blast, Bacterial Leaf Blight, Narrow Brown Spot, and Brown Spot.
- **Prediction confidence:** Recording the confidence scores associated with model predictions to understand the certainty of individual detections.
- **Detection performance:** Evaluating the correctness of predicted classes and, where bounding-box detection is implemented, the overlap between predicted and ground-truth regions.

The model can be evaluated using precision, recall, F1-score, and confusion matrices for classification tasks. If the implementation uses bounding-box object detection, Intersection over Union (IoU) and mean Average Precision (mAP) can also be used. Accuracy may be reported where the task and test dataset support a meaningful accuracy calculation.

The validation results should be recorded for each disease class as well as for the overall model. This helps identify diseases that are detected reliably and classes that require additional training data or model improvement.

2.2.2 Hybrid Framework Validation

The hybrid framework validation evaluates the complete workflow rather than examining the model in isolation. In this component, the term hybrid framework refers to the combination of sequential image analysis, disease identification, affected-area-based severity estimation, and system integration. It does not imply a separate multimodal fusion model unless one is implemented.

The framework is validated by submitting different types of images and verifying that the system follows the intended processing sequence.

The expected workflow is:

1. The user uploads an image.
2. The system checks whether the image contains a paddy leaf.
3. If the image is valid, the system determines whether the leaf is healthy or unhealthy.
4. If the leaf is unhealthy, the system identifies the disease as Rice Blast, Bacterial Leaf Blight, Narrow Brown Spot, or Brown Spot.
5. The system estimates disease severity based on the affected leaf area.
6. The system displays the relevant prediction results, including disease name, estimated severity, and prediction confidence score.

The following validation scenarios should be considered.

Table 2.6. Test Scenario Validation

Test scenario	Expected outcome
Valid image of a healthy paddy leaf	The system identifies a paddy leaf and reports it as healthy.
Image of a leaf affected by	The system identifies the disease as Rice Blast and estimates

Rice Blast	severity.
Image of a leaf affected by Bacterial Leaf Blight	The system identifies the disease as Bacterial Leaf Blight and estimates severity.
Image of a leaf affected by Narrow Brown Spot	The system identifies the disease as Narrow Brown Spot and estimates severity.
Image of a leaf affected by Brown Spot	The system identifies the disease as Brown Spot and estimates severity.
Image that does not contain a paddy leaf	The system rejects the image or displays an appropriate validation message.
Invalid or unsupported image file	The system displays an error message without crashing.
Image containing visible disease spread	The estimated severity reflects the measured affected leaf area according to the implemented calculation method.

The validation process should compare system outputs with manually verified labels and affected area annotations, where available. This is particularly important for severity estimation because a correct disease label does not necessarily mean that the severity estimate is accurate.

The complete framework should also be tested for correct data transfer between the user interface and backend, consistent output formatting, and appropriate handling of failed predictions.

2.3 Project Requirements

2.3.1 Functional Requirements

Functional requirements define the operations that the paddy disease detection component must perform to meet its intended objectives.

The main functional requirements are as follows.

Table 2.7. Functional Requirements

ID	Functional requirement	Description
FR01	Image Upload	The system shall allow users to upload paddy leaf images for analysis.
FR02	Image Validation	The system shall determine whether the uploaded image contains a paddy leaf.
FR03	Health Status Identification	The system shall identify whether a valid paddy leaf is healthy or unhealthy.
FR04	Disease Detection	The system shall identify the relevant disease in an unhealthy paddy leaf.
FR05	Multiple Disease Classification	The system shall support Rice Blast, Bacterial Leaf Blight, Narrow Brown Spot, and Brown Spot.
FR06	Severity Estimation	The system shall estimate disease severity based on the affected area of the leaf.
FR07	Prediction Confidence	The system shall display the confidence score associated with the model prediction.
FR08	Result Display	The system shall display the detected disease, health status, severity estimate, and relevant prediction details.
FR09	Error Handling	The system shall display suitable error messages for invalid images, unsupported file types, and unsuccessful prediction requests.
FR10	Backend Integration	The system shall communicate with the backend to process

		uploaded images and return analysis results.
FR11	Input Verification	The system shall check uploaded files for supported formats and acceptable file sizes before processing.
FR12	Result Consistency	The system shall present results in a consistent format for different supported disease classes.

2.3.2 Non-Functional Requirements

- **Performance:** The system should process uploaded paddy leaf images and return prediction results within an acceptable response time.
- **Accuracy and Reliability:** The system should provide reliable paddy leaf validation, health classification, disease detection, and severity estimation.
- **Usability:** The user interface should be simple, clear, and easy for farmers and agricultural officers to use.
- **Scalability:** The system should support future increases in users, image uploads, and dataset size.
- **Security:** The system should validate uploaded files and protect the application and backend against unauthorized access.
- **Maintainability:** The system should allow future updates to the YOLOv8 model, disease classes, and application features.

2.3.3 System Requirements

System requirements specify the hardware and software resources needed to develop, train, test, and operate the paddy disease detection component.

The component requires computing resources for model development, image processing, backend execution, and system testing. The actual hardware requirements depend on the YOLOv8s configuration, image resolution, model-training approach, and deployment environment.

The following table presents requirements rather than verified specifications of the system implementation.

Table 2.8. Hardware Components

Hardware component	**Suggested requirement**	**Purpose**
Processor	Modern multi-core CPU	Backend execution, image processing, and general development.
RAM	16 GB	Running development tools, processing images, and executing the application.
Storage	At least 10 GB of available space, with additional space as required	Storing datasets, model weights, source code, and test results.
GPU	CUDA-compatible NVIDIA GPU for local GPU-based training, if applicable	Accelerating YOLOv8 model training and inference.
Camera or smartphone	Camera capable of capturing clear paddy leaf images	Collecting images for testing and practical use.
Display device	Laptop, desktop, tablet, or smartphone	Accessing the system and viewing prediction results.
Network connection	Internet or local network connectivity, depending on deployment	Connecting to the hosted application or backend service.

The software requirements include the tools, libraries, and platforms used to implement the detection component.

Table 2.9. Software Components

Software component	**Purpose**
Python	Implementing the machine learning and image analysis logic.
Ultralytics YOLOv8	Developing and executing the YOLOv8-based paddy leaf analysis model.
PyTorch	Providing the deep learning framework used by the YOLOv8 implementation.

OpenCV	Supporting image loading, processing, and visualization where required.
NumPy	Supporting numerical operations and image-related calculations.
Flask or the actual backend framework used	Receiving image requests and returning prediction results, if applicable.
REST API tools, such as Postman	Testing backend endpoints and verifying request-response behaviour.
Frontend development tools	Developing the interface for image upload and displaying analysis results.
Git and GitHub, if used	Supporting source-code version control and collaboration.
Web browser	Accessing and testing the web-based application.

2.4 Ethical and Security Considerations

Ethical and security considerations are important when developing an AI-based agricultural disease detection system. The component processes agricultural images and generates predictions that may influence how farmers understand the health of their paddy crops. Therefore, the system should be developed and evaluated with attention to data privacy, prediction reliability, security, and responsible use.

2.4.1 Data Privacy and Protection

The system should process uploaded images only for the intended disease analysis purpose. If images contain personal information, identifiable individuals, or location details, such information should be protected and should not be collected unnecessarily.

Access to stored images, datasets, prediction records, and model files should be restricted to authorized users. If images are retained for future research or model improvement, users should be informed about the purpose of storage and the applicable data-handling practices..

2.4.1 Data Privacy and Protection

The web application and backend should include appropriate security controls to prevent unauthorized access, misuse, and malicious file uploads. Important measures include:

- Validating uploaded file types and file sizes.
- Rejecting corrupted or unsupported image files.
- Restricting access to sensitive backend endpoints where necessary.
- Avoiding the exposure of credentials, API keys, and internal system information.
- Using HTTPS when transmitting images and prediction results over public networks.
- Applying suitable error handling without exposing sensitive technical details.
- Keeping software dependencies and deployment configurations updated.

2.5 Commercialization Aspects of the Product

The paddy disease detection component has potential for commercialization as part of a smart agriculture platform designed to assist farmers in identifying paddy leaf diseases and understanding the extent of visible infection.

The component uses a YOLOv8-based image analysis workflow to identify paddy leaves, distinguish healthy and unhealthy leaves, detect four target diseases, and estimate disease severity based on the affected leaf area. These functions could support the development of a practical digital agricultural service for farmers and agricultural support organizations.

However, the current component should be considered a research prototype until its performance, reliability, usability, and operational costs have been validated under real cultivation conditions.

2.5.1 Target Audience

The potential target audience includes the following groups.

1. Paddy Farmers

Small-scale and medium-scale paddy farmers in Sri Lanka could use the system to examine paddy leaf images, identify possible diseases, and understand the estimated severity of visible symptoms. A simple mobile-friendly interface could make the technology more accessible to farmers.

2. Agricultural Extension Officers

Agricultural extension officers could use the component as an additional screening tool when assessing reported crop diseases and providing preliminary support to farmers. The output may help prioritize cases that require further field inspection.

3. Agricultural Researchers and Educational Institutions

Universities, agricultural research institutes, and students could use the component to study AI-based plant disease detection, severity estimation, and computer vision applications in agriculture.

4. Agritech Companies and Agricultural Service Providers

Agricultural technology companies could integrate the disease analysis component into existing crop-monitoring platforms, digital farming applications, or agricultural advisory services, subject to technical validation and licensing arrangements.

2.5.2 Market Space

The potential market space for this component is the digital agriculture and AI-assisted crop monitoring sector in Sri Lanka. The product addresses the need for accessible tools that help users identify common paddy leaf diseases and assess visible infection severity.

The main market opportunities are:

- **Direct farmer access:** Providing an easy-to-use web application through which farmers can upload paddy leaf images and receive disease analysis results.
- **Institutional use:** Supporting agricultural extension services, research organizations, and educational institutions in preliminary disease screening and demonstration activities.
- **Integration with agricultural platforms:** Offering the detection component as an API or software module that can be integrated into a wider agricultural Decision Support System.
- **Research and technology partnerships:** Collaborating with agricultural institutions and agritech developers to improve datasets, validate model performance, and expand system capabilities.

The actual commercial opportunity will depend on model accuracy in field conditions, access to reliable agricultural datasets, farmers' digital literacy, internet availability, affordability, and the availability of alternative agricultural advisory services.

2.5.3 Potential Revenue Models

Table 2.10. Revenue Models

Revenue model	Description
Freemium access	Offer basic disease detection free of charge and consider paid advanced features after validation.
Institutional licensing	Provide the software to agricultural institutions or organizations under an agreed licensing model.
API-based access	Allow other agricultural applications to access the disease detection service through a controlled API.
Subscription plans	Offer optional subscription packages for organizations or professional users requiring additional services.
Research partnerships	Collaborate with universities and agricultural organizations on funded research and system development.

These are potential commercialization strategies rather than existing revenue streams. Pricing and demand should be evaluated through market research and discussions with potential users.

The component's potential competitive advantages include:

- Localized application: The disease classes are selected to address paddy cultivation needs relevant to Sri Lanka.
- Sequential analysis: The system checks paddy leaf validity and health status before proceeding to disease identification.
- Severity estimation: It aims to provide information about the affected leaf area instead of reporting only a disease label.
- Digital accessibility: A web-based interface can make the analysis process accessible without requiring users to install specialized desktop software.
- Integration potential: The component can serve as a disease analysis module within the wider R26-IT-066 smart agriculture Decision Support System.

These advantages should be confirmed through comparison with existing solutions and testing with representative users.

2.5.4 Commercialization Challenges

Several challenges must be addressed before the component can be deployed commercially. These include collecting sufficiently diverse data set, validating model performance in real paddy fields, improving detection reliability for visually similar diseases, ensuring accurate severity estimation, and maintaining the backend infrastructure.

Overall, the component provides a foundation for a potential AI-assisted paddy disease screening product. Further field validation, user feedback, security testing, and operational cost analysis are necessary before determining its commercial readiness.

3. RESULTS AND DISCUSSION

3.1 Results

The paddy leaf disease detection component was developed using the YOLOv8 object-detection model to identify visual patterns associated with paddy leaf conditions. The evaluation outputs include dataset distribution plots, training and validation performance curves, confidence-based performance curves, precision–recall analysis, confusion matrices, and example images containing predicted bounding boxes.

The results indicate that the trained model learned to identify several visual classes in the dataset and validation metrics, precision–recall analysis, confidence curves, confusion matrices, and visual examples of predicted disease regions. However, the performance differs between classes, with leaf_blast and brown_spot showing lower Average Precision values than some of the other displayed classes. The model was trained for 50 epochs using yolov8n.pt, an input image size of 640 × 640 pixels, and a batch size of 16. The training configuration specifies CPU execution and validation during training.

The following subsections describe the available evidence and discuss its implications for the implemented component.

3.1.1 Dataset Results

Table 3.1. Data set Results

Class	**Actual Result**
Total images	957

Rice Blast images	183
Brown Spot images	194
Healthy images	201
Leaf Blast images	178
Narrow Brown images	178
Leaf Scald images	23

The dataset contains 957 annotated instances across six displayed classes. The healthy class has the largest number of instances (201), followed by brown spot (194). Leaf scald has the fewest instances (23), indicating a considerable imbalance between the represented classes.

The difference in class representation is important when interpreting the model's performance. A class with fewer examples may provide less visual variety during training and may be more difficult to detect reliably. In this dataset, the small number of leaf-scald instances could limit the generalization of predictions for that class. The effect cannot be confirmed from the instance counts alone, and further evaluation using class-specific test results would be needed.

The dataset visualization also shows the spatial distribution and dimensions of the annotated bounding boxes. These plots help describe how objects are positioned and sized within the training images. Such information is relevant to an object-detection model because the model must learn to recognize disease patterns across different locations and sizes.

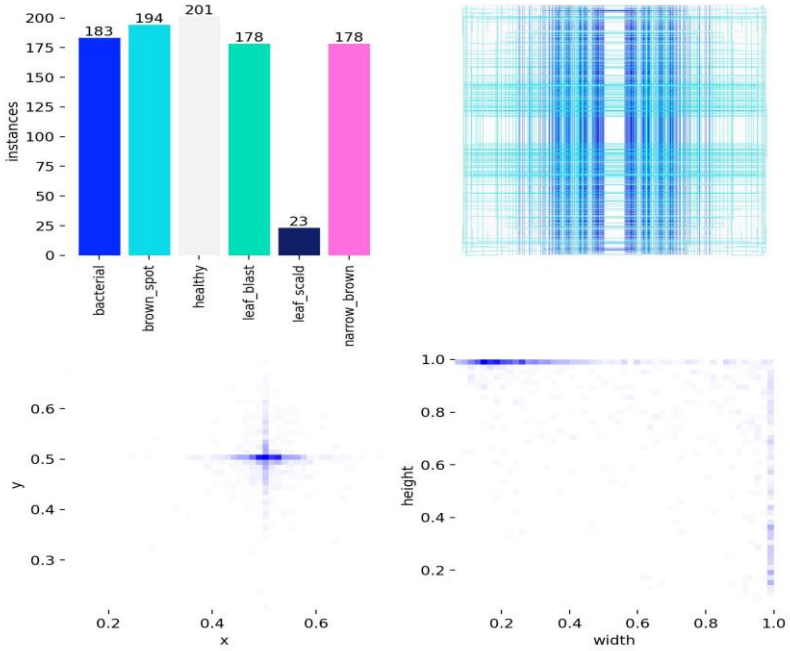


Figure 3.1: Class distribution and bounding-box characteristics of the paddy leaf dataset.

3.1.2 YOLOv8s Detection Results

The training-results figure presents the changes in bounding-box loss, classification loss, distribution focal loss, precision, recall, mAP@0.5, and mAP@0.5:0.95 over approximately 50 training epochs.

The training bounding-box loss decreases from approximately 1.2 to 0.9, while the classification loss decreases from approximately 3.0 to 0.6. The distribution focal loss also shows an overall decreasing trend. These changes indicate that the model progressively reduced its training losses during optimization.

The validation losses fluctuate more than the training losses, particularly the validation bounding box and distribution focal losses. Nevertheless, the validation classification loss generally decreases over the observed epochs. Some fluctuations remain, so the curves alone do not establish that the model has fully converged or that overfitting is absent.

The detection metrics increase over the training period. The precision curve finishes at approximately 0.92, and the recall curve finishes at approximately 0.93. The mAP@0.5 curve reaches approximately 0.95, while mAP@0.5:0.95 reaches approximately 0.59, based on visual readings of the uploaded graph. These are approximate values from the plotted curves; the exact values should be taken from the final evaluation output if available.

The difference between mAP@0.5 and mAP@0.5:0.95 is noteworthy. The first metric evaluates detections at an IoU threshold of 0.5, while the second averages performance over a stricter range of IoU thresholds. The lower mAP@0.5:0.95 suggests that localization performance is less consistent when stricter bounding-box overlap requirements are applied.

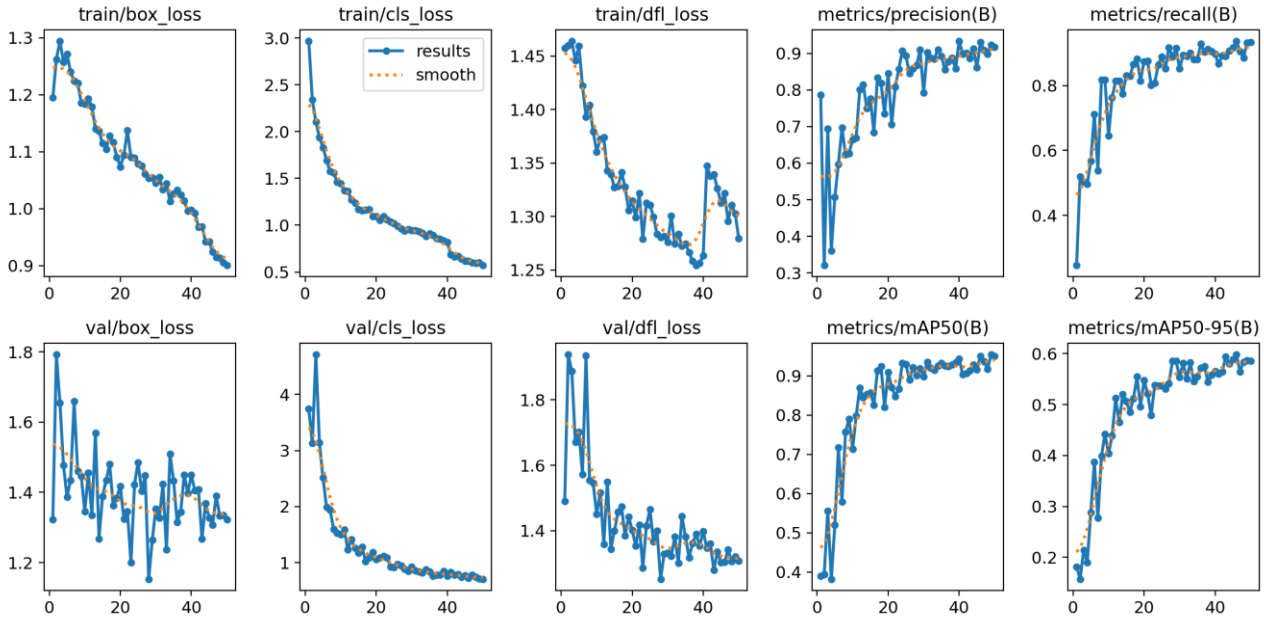


Figure 3.2: YOLOv8s training and validation metrics across training epochs.

The precision–recall curve illustrates the relationship between precision and recall for each class as the detection threshold changes. The legend in the uploaded figure reports the following Average Precision (AP) values.

Table 3.2: Per-class AP values

Class shown in figure	**AP at IoU 0.5**
Bacterial	0.995
Brown spot	0.904
Healthy	0.974
Leaf blast	0.885
Narrow brown	0.993
All classes (mAP@0.5)	0.950

The model demonstrates its highest reported AP for bacterial disease (0.995), followed by narrow brown (0.993) and healthy leaves (0.974). These values indicate strong precision–recall performance for those classes under the evaluation represented by the figure.

Brown spot has an AP of 0.904, while leaf blast has the lowest reported AP of 0.885. This indicates that the model's performance is not uniform across the displayed categories. Leaf blast may require additional investigation through more diverse training samples, review of annotation quality, and class-specific error analysis.

The overall mAP@0.5 of 0.950 represents the mean of the displayed class AP values. It is evidence of strong object-detection performance on the evaluation represented by the graph, but it should not be interpreted as a guarantee that 95% of all user-submitted images will receive the correct disease label.

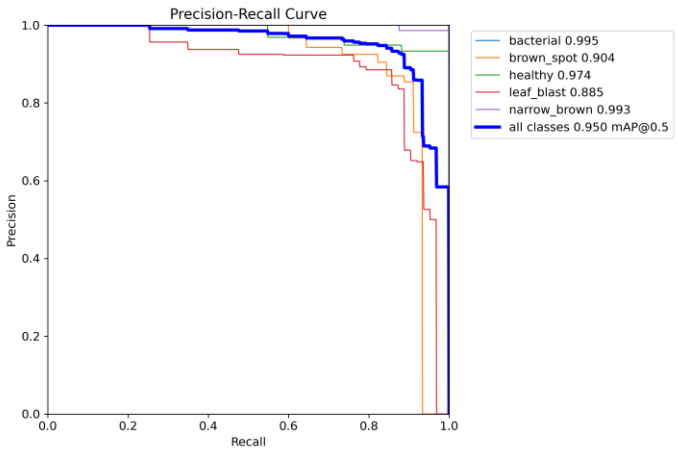


Figure 3.3: Precision–recall curves for the displayed paddy leaf classes.

The confidence curves show how precision, recall, and F1-score change when different prediction-confidence thresholds are applied.

The F1-confidence curve reports a maximum overall F1-score of 0.93 at a confidence threshold of 0.464. The F1-score combines precision and recall into a single measure, making it useful when both false-positive and false-negative predictions matter.

The precision-confidence curve reports a precision value of 1.00 at a threshold of 0.899. Although this is the maximum shown in the graph, a high confidence threshold can reject more predictions and reduce recall. Therefore, selecting a threshold based on precision alone may cause genuine disease regions to be missed.

The recall-confidence curve reports a maximum value of approximately 0.98 at a threshold of 0.000. This reflects the tendency for recall to be higher when predictions are accepted at very low confidence thresholds. However, accepting low-confidence predictions may also introduce more false positives.

Together, these curves demonstrate the trade-off between precision and recall. The threshold should be selected according to the intended use of the component and validated against an independent evaluation dataset. The best threshold for the plotted F1-score is not necessarily the same as the threshold currently used by the application.

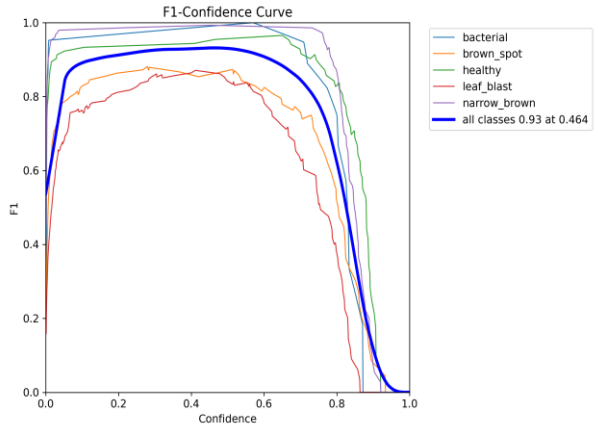


Figure 3.4: F1-score against confidence threshold.

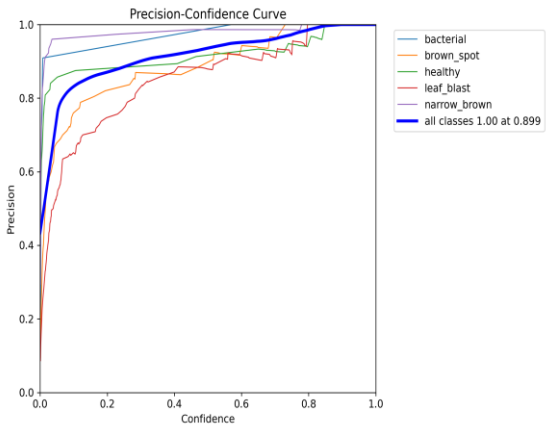


Figure 3.5: Precision against confidence

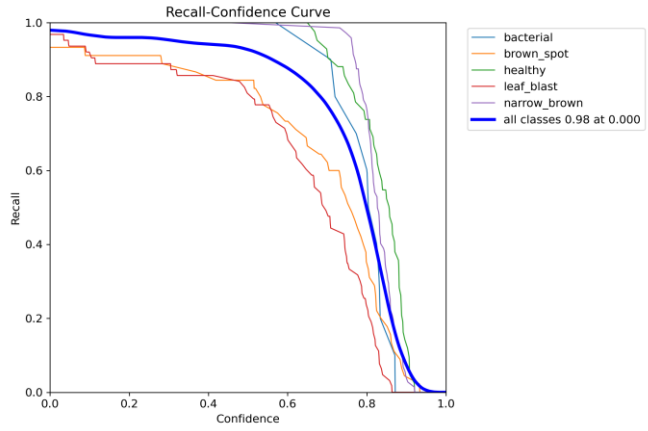


Figure 3.6: Recall against confidence threshold.

The uploaded confusion matrix and its normalised version provide additional evidence about the model's class-level predictions. In the displayed matrix, the vertical axis represents predicted classes, while the horizontal axis represents true classes. Values along the diagonal represent correct

class assignments, while off-diagonal values indicate confusion between classes or background-related detection errors.

The largest visible diagonal counts include 72 for narrow brown, 53 for leaf blast, 41 for brown spot, 41 for healthy, and 10 for bacterial. The normalised matrix shows diagonal values of 1.00 for bacterial, 0.91 for brown spot, 0.98 for healthy, 0.84 for leaf blast, and 1.00 for narrow brown.

These results suggest that leaf blast has comparatively lower class-level performance in this matrix than the other listed classes. The matrix also contains off-diagonal entries, including brown spot instances predicted as leaf blast and healthy instances predicted as leaf blast. Such errors may occur when visual patterns overlap or when image quality, background, or annotation differences make the classes difficult to distinguish.

The background column contains multiple detections, including 16 under predicted leaf blast, suggesting that background-related false positives are an important error category in this evaluation. The background row also includes missed detections for some true classes. These results indicate that improving the distinction between disease patterns and non-disease regions could be useful.

There is a limitation in interpreting these matrices: they include leaf scald and background, while the uploaded precision–recall graph lists five classes and does not include leaf scald. Before the thesis is finalized, the matrix and graph should be checked against the same model version, dataset split, and class mapping. The reported counts should not be combined with results from a different evaluation run.

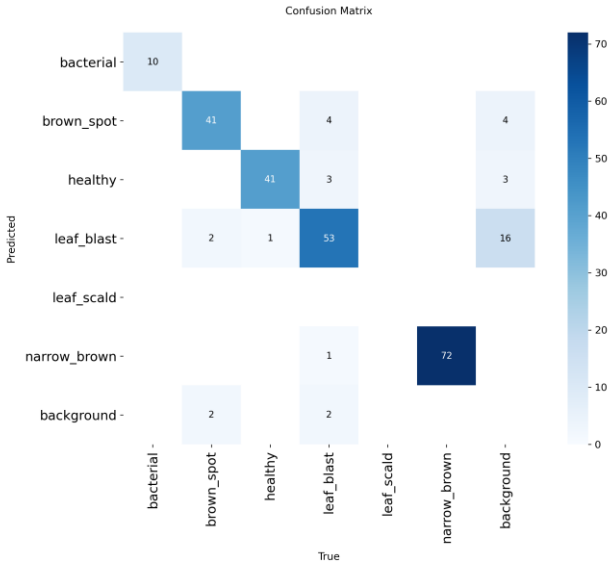


Figure 3.7: Confusion matrix of YOLOv8s class predictions.

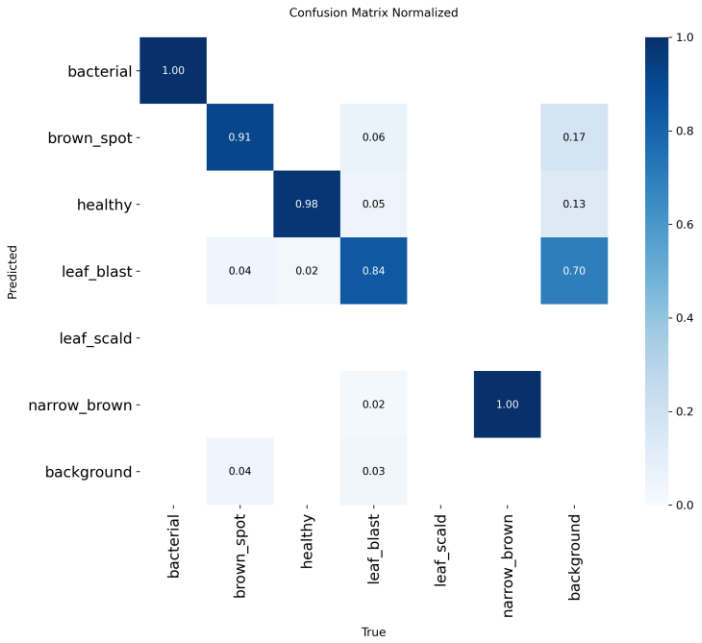


Figure 3.8: Normalized confusion matrix of YOLOv8s class predictions.

3.1.3 Integrated Detection–Classification Pipeline Results

The YOLOv8 bounding boxes around visually identified regions in paddy leaf images. The annotated images include examples associated with bacterial disease, brown spot, leaf blast, healthy leaves, and narrow brown disease. The bounding boxes and class labels provide a visual representation of the regions identified by the model.

The model can generate localized detections on different leaf orientations, sizes, and backgrounds. Some images contain long, narrow leaves, while others contain smaller visible disease regions. This variation is relevant because the model identifies disease-related visual patterns even when their scale and position differ.

However, the prediction examples also show why visual inspection remains important. Some bounding boxes cover a large part of a leaf, whereas others cover smaller regions. A predicted bounding box identifies an estimated region associated with a class; it does not necessarily outline the exact pixels affected by the disease. Similarly, a displayed class label represents the model's prediction rather than independent confirmation of the actual disease. The figure provides evidence of detection behaviour, but they do not independently establish the model's accuracy on unseen images.

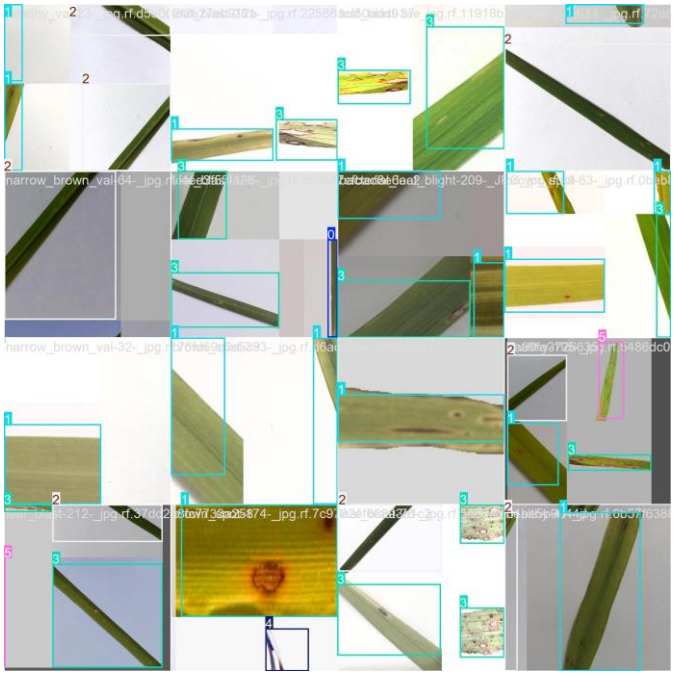


Figure 3.9: Examples of YOLOv8s predictions on paddy leaf images.

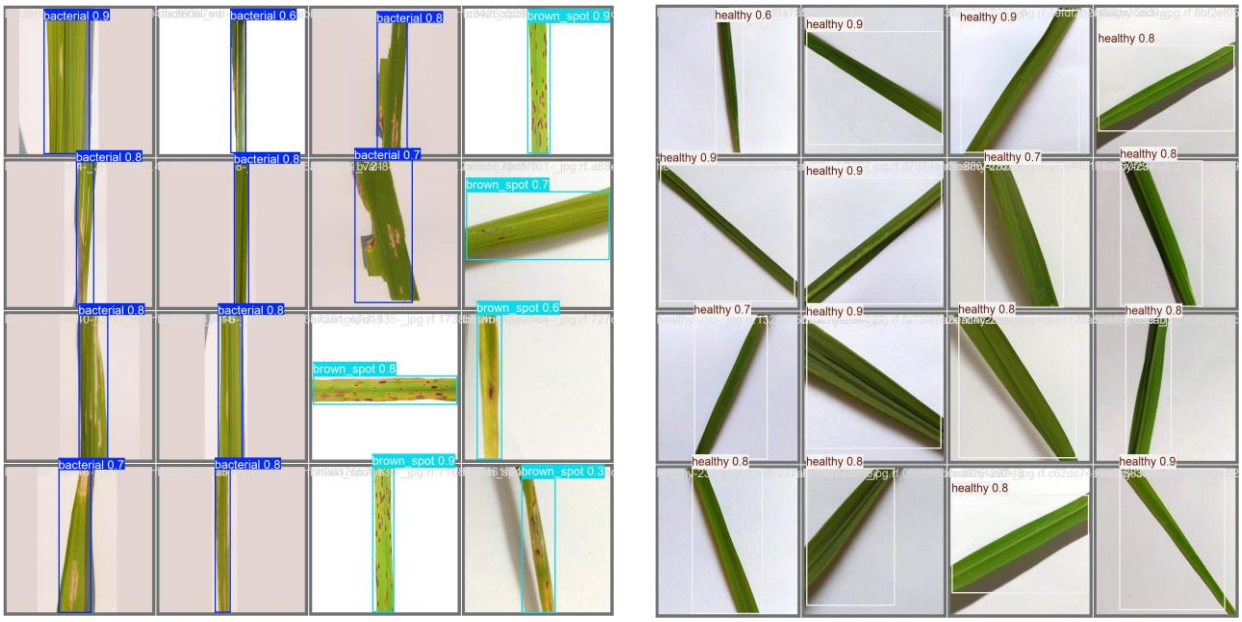


Figure 3.10: Example disease detections with predicted class labels and bounding boxes.

The integrated component combines a Flutter interface with a Flask API and a YOLOv8 model. The user selects a leaf image through the application, and the image is submitted to the backend using the /disease/predict endpoint. The backend reads the image and passes it to the trained model for inference.

Following inference, the backend extracts the predicted class, confidence score, and bounding-box coordinates. The detected box areas are then used to estimate the spread percentage, and the

resulting percentage is mapped to a severity category. The prediction information is returned to the frontend as a JSON response for display.

The implemented spread calculation can be expressed as:

$$\text{Spread (\%)} = \frac{\sum_{i=1}^{n} A_i}{A_{\text{image}}} \times 100$$

where (A_i) represents the area of a detected bounding box and (A_{image}) represents the total image area.

The implementation assigns severity categories using the following thresholds:

Table 3.3. Severity Grades

Estimated spread	**Severity category**
0–5%	Mild
Above 5–20%	Moderate
Above 20–40%	High
Above 40%	Severe

These categories represent the thresholds implemented in the backend. Because the calculation uses bounding-box areas, overlapping boxes can contribute to an overestimate, and the bounding boxes may include healthy pixels. Consequently, the estimated spread should be interpreted as a bounding-box coverage estimate rather than a precise measurement of diseased leaf tissue.

The integrated pipeline therefore combines object detection with a rule-based severity estimate. The disease label and confidence score originate from model inference, whereas the severity category is derived from the estimated spread percentage. These outputs have different meanings and should be evaluated separately.

The confidence score represents the model's confidence in a prediction. It is not equivalent to the model's overall accuracy. A prediction with high confidence can still be incorrect, particularly when an input differs from the training data or contains visual patterns shared by multiple classes.

The confidence curves demonstrate that the balance between precision and recall changes with the selected threshold. Therefore, the confidence threshold used in the deployed application should be

documented separately from the threshold associated with the best F1-score in the evaluation graphs.

The implementation also requires careful interpretation of healthy and non-leaf predictions. Returning a healthy result when no detection is found does not, by itself, prove that the uploaded image is a healthy paddy leaf. It may also indicate that the model did not detect a recognised region. A dedicated paddy-leaf validation mechanism would be required to reliably distinguish unrelated images from valid paddy leaf images.

The model's displayed classes must also be reconciled with the component's stated requirements. The target disease categories are rice blast, bacterial leaf disease, brown spot, and narrow brown spot, together with healthy leaves. However, the evaluation figures use labels such as leaf_blast, bacterial, brown_spot, narrow_brown, and leaf_scald. The final report should verify the class names in the trained model's configuration and confirm that they map correctly to the disease names displayed in the application.

3.1.4 Integration Results

The report demonstrates whether the disease component successfully provides outputs that can be consumed by the wider R26-IT-066 system. Evidence includes screenshots from the client application and, where applicable, the harvest estimation component receiving disease information.

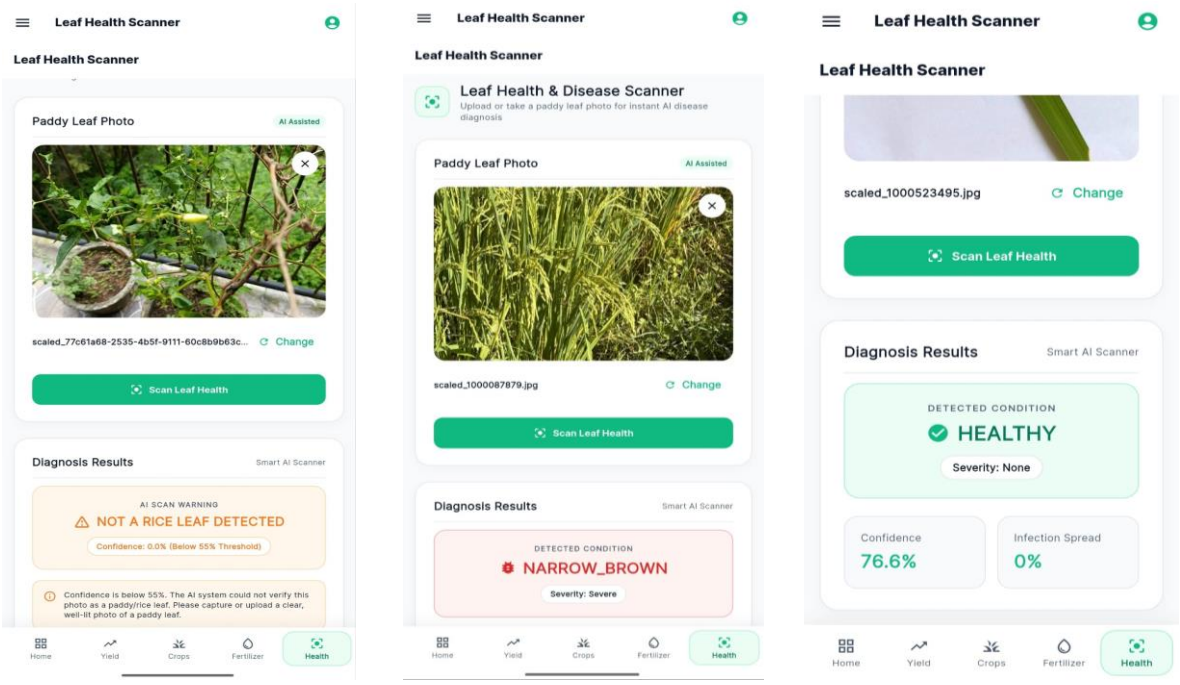


Figure 3.11. Detection page with image upload, disease results with severity grades.

The API's output has four key values for interpretation: the predicted disease name, the confidence percentage, the estimated spread-area percentage, and the severity label. The confidence percentage represents the model's confidence for the selected detection; it should not be described as the model's overall accuracy. Accuracy is a dataset-level evaluation statistic calculated by comparing predictions with ground-truth labels across many examples.

The spread percentage is based on the area of predicted rectangles relative to the whole image. Because the code sums all box areas without removing overlaps or restricting the denominator to the leaf area, the result is an approximate image-coverage value. It should not be equated directly with the percentage of diseased leaf tissue. The labels Mild, Moderate, High, and Severe are currently generated by hard-coded thresholds. Before the system is used to guide real interventions, those thresholds should be validated against expert-rated examples.

3.2 Research Findings

The principal verified finding is that the component has been integrated as a functioning inference pathway: a Flutter screen sends a leaf image to a Flask endpoint, YOLOv8 inference returns a class and confidence, and the backend calculates a bounding-box coverage estimate and severity label. The interface displays these values in a farmer-facing format.

A second finding is that the current implementation has several interpretation risks that materially affect the reliability of its output. Most notably, a low-confidence result is converted to healthy in the backend, the UI and backend thresholds differ, and the spread estimate is based on bounding rectangles rather than a lesion segmentation mask. These are not merely documentation issues; they should be treated as validation and implementation tasks.

A model-performance conclusion cannot yet be made because the supplied files do not establish an independent test-set evaluation. The final dissertation should report measured values only after running a reproducible evaluation and retaining the prediction logs, labels, metric output, and confusion matrix.

3.3 Summary of Each Student's Contribution

The R26-IT-066 Smart Agriculture Decision Support System consists of four individual components. Each component contributes a specific function to the overall system.

1. IT22178718 – IoT / Field Monitoring

This component focuses on collecting field and environmental information to support agricultural monitoring and decision-making. The information supplied by this component provides relevant field context for the wider smart agriculture system.

2. IT21811104 – Crop Recommendation / Market Analysis

This component focuses on providing crop-related and market-related analysis to support agricultural planning. Its purpose is to help connect cultivation decisions with relevant crop and market information within the wider system.

3. IT22261632 – Paddy Leaf Disease Detection and Severity Analysis

This component focuses on analysing paddy leaf images to identify disease conditions and estimate their severity. Its intended workflow includes paddy-leaf validation, healthy or unhealthy classification, identification of Rice Blast, Bacterial Leaf Blight, Brown Spot, and Narrow Brown Spot, and estimation of disease severity based on the affected area. The component also provides prediction confidence and disease-related output through the backend API. These outputs are intended to support disease-aware decision-making and provide information to the harvest estimation component.

4. IT22266446 – Disease-Aware Harvest Estimation

This component focuses on estimating harvest or yield-related impacts using disease severity and other relevant inputs. It uses disease-related information from the disease detection component as part of the wider harvest estimation and decision-support process.

Overall Contribution

Together, these four components form an integrated smart agriculture system. The IoT component provides field information, the crop recommendation component contributes crop and market analysis, the disease detection component provides paddy disease and severity information, and the harvest estimation component uses relevant inputs to support harvest-related decisions. The integration of these components aims to provide a more comprehensive decision-support solution for agricultural activities.

3.4 Discussion

The paddy leaf disease detection and severity analysis component aims to identify diseases from paddy leaf images and provide information that supports agricultural decision-making. The component forms part of the R26-IT-066 Smart Agriculture Decision Support System and focuses on analyzing leaf images, identifying disease conditions, estimating disease severity, and providing prediction results through the integrated application.

The implemented workflow uses a disease prediction backend to process an uploaded image and return disease-related information. The component is intended to identify Rice Blast, Bacterial Leaf Blight, Brown Spot, and Narrow Brown Spot, together with the prediction confidence and an estimate of disease severity based on the affected area. These outputs can support the wider system, particularly the disease-aware harvest estimation component, which uses disease-related information as an input for its analysis.

An important aspect of the component is that it extends disease identification beyond simply returning a disease name. By including severity-related information, it aims to provide a more useful understanding of the potential condition of the paddy leaf. However, the affected-area calculation should be considered an estimate rather than a precise measurement of diseased tissue. More accurate severity assessment would require further validation using appropriately labelled images and expert assessments.

The reliability of the system depends on the quality and diversity of the image dataset, the accuracy of the annotations, and the model's ability to recognize disease symptoms under different conditions. Factors such as lighting, background complexity, image blur, leaf orientation, and partially visible symptoms may influence prediction results. Therefore, the model should be evaluated using a separate test set, and the results should be reported using appropriate performance metrics and a confusion matrix.

Although the available project documentation confirms the disease prediction workflow and its backend integration, verified final test-set metrics have not been provided in the available thesis documents. Consequently, specific claims about accuracy, precision, recall, F1-score, and overall reliability should only be made after completing the final evaluation.

Overall, the disease detection and severity analysis component contributes an image-based disease assessment capability to the wider smart agriculture system. Future improvements should focus on

validating the model using an independent test dataset, improving the consistency of prediction thresholds, validating severity estimates, and assessing performance under real field conditions.

4. CONCLUSION AND FUTURE WORK

4.1 Conclusion

This research addresses the need for disease-aware decision support in Sri Lankan paddy cultivation. The IT22261632 component is positioned within the R26-IT-066 Smart Agriculture Decision Support System as the module responsible for paddy disease detection and severity analysis.

The research proposal defines a localized approach based on paddy leaf images, with Rice Blast and Brown Spot as the principal diseases and a four-level severity representation. It also proposes the use of deep-learning models and a management recommendation layer to make disease analysis more actionable.

The available implementation evidence confirms that the integrated project contains a dedicated disease module, a trained model artifact and a /disease/predict API endpoint. The final evaluation must determine the actual performance of the implemented model and the extent to which the proposed severity and recommendation features have been realized.

4.2 Future Work

• Expand the dataset with more Sri Lankan field images covering diverse lighting, backgrounds, growth stages and disease presentations.

• Improve severity estimation using stronger annotation protocols and balanced severity classes.

• Evaluate alternative deep-learning architectures and compare them using a consistent held-out test set.

• Strengthen environmental-data integration so that disease analysis can be interpreted alongside current field conditions.

• Strengthen authentication, monitoring and deployment controls before production use.

• Conduct larger-scale field validation with agricultural experts and farmers.

5.References

[1] Department of Census and Statistics Sri Lanka, “Census of Agriculture 2021: Paddy Farming Households,” Colombo: Ministry of Finance, 2022.

[2] International Rice Research Institute (IRRI), “Compendium of Rice Diseases and Pests,” 2nd ed., Los Baños, Philippines: IRRI Press, 2014.

[3] Department of Agriculture Sri Lanka, “Annual Performance Report 2023: Crop Losses and Disease Impact Assessment,” Peradeniya, 2023.

[4] D. Pathirana and W. Wijesinghe, “Agricultural extension officer-to-farmer ratio and technology adoption gaps in rural Sri Lanka,” Journal of Agricultural Sciences, Sri Lanka, 2020.

[5] Department of Agriculture Sri Lanka, “Crop Recommendations and Fertilizer Guidelines,” Peradeniya, 2021.

[6] S. P. Mohanty, D. P. Hughes, and M. Salathe, “Using deep learning for image-based plant disease detection,” Frontiers in Plant Science, 2016.

[7] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” Computers and Electronics in Agriculture, 2018.

[8] Ultralytics, “YOLOv8,” Ultralytics documentation. [Online]. Available: https://docs.ultralytics.com/models/yolov8/. Accessed: Oct. 10, 2026.

[9] Plantix, “Plantix: Crop diagnosis and treatment recommendations.” [Online]. Available: https://plantix.net/. Accessed: Oct. 10, 2026.

[10] Y. Wang, U. S. R. Dhamodharan, N. Sarwar, F. A. Almalki, and D. Mohan, “A hybrid approach for rice crop disease detection in agricultural IoT system,” Discover Sustainability, 2024.

[11] H. D. Jayasinghe and T. Ranasinghe, “Convolutional neural network-based paddy leaf disease classification under Sri Lankan field conditions,” Ceylon Journal of Science, 2021.

[12] A. S. Perera, L. H. P. Karunaratne, and W. Senevirathne, “Transfer learning for rice blast detection using locally collected Sri Lankan field imagery,” Sri Lanka Journal of Food and Agriculture, 2022.

[13] R. Amarasinghe, C. Jayawardena, and K. Bandara, “CNN-LSTM hybrid models for temporal disease progression forecasting in tropical paddy cultivation,” Sri Lanka Journal of Information Technology, 2022.

[14] D. M. S. N. Dissanayake and K. P. Wijesinghe, “IoT-based smart agriculture framework for paddy cultivation in Kurunegala district,” Journal of the National Science Foundation of Sri Lanka, 2020.

[15] P. G. Wickramasinghe, R. M. T. C. Rajapaksha, and J. M. J. W. Jayasinghe, “Harvest loss patterns for rice blast and brown spot across Maha and Yala seasons in the North Central Province of Sri Lanka,” 2023.

[16] Bayer Crop Science, “Plantix, AI-based plant disease diagnosis,” 2024.

[17] Department of Agriculture Sri Lanka, “e-Krushi Agricultural Advisory Portal,” 2024.